

UNIVERSITY OF BUEA
COLLEGE OF TECHNOLOGY (COLTECH)
DEPARTMENT OF COMPUTER ENGINEERING



A Project Report submitted in partial fulfilment of the requirements for the award of the

Bachelor of Technology (B.Tech) in Software Engineering

TOPIC

**Design and Implementation of LexManage: A Multi-Tenant SaaS
Legal Practice Management Platform with AI-Powered Document
Analysis**

Presented by: **Emmanuel Nyouma**

Supervised by: **Mr Baloko**

Academic Year 2025/2026

Buea, Cameroon

ATTESTATION

I, **Emmanuel Nyouma**, hereby declare that this project report entitled “Design and Implementation of LexManage: A Multi-Tenant SaaS Legal Practice Management Platform with AI-Powered Document Analysis” is the result of my own original work carried out under the supervision of Mr Baloko. To the best of my knowledge, it contains no material previously published or written by another person, nor material which has been accepted for the award of any other degree or diploma, except where due acknowledgement has been made in the text. All sources of information have been duly referenced.

Student: **Emmanuel Nyouma**

Signature:

Date:

Supervisor: **Mr Baloko**

Signature:

Date:

DEDICATION

To my family, whose constant encouragement and sacrifices made this work possible, and to every legal professional striving to serve justice more efficiently.

ACKNOWLEDGEMENTS

I wish to express my sincere gratitude to my supervisor, **Mr Baloko**, for the guidance, constructive criticism and patience that shaped this project from concept to completion.

My thanks also go to the academic staff of the Department Of Computer Engineering, University Of Buea, for the solid grounding in software engineering principles that underpins this work.

Finally, I am deeply grateful to my family, friends and classmates for their moral support, and to the open-source community whose tools made this platform achievable.

ABSTRACT

Law firms manage a high volume of cases, clients, hearings and confidential documents, yet many still rely on fragmented tools—spreadsheets, shared drives and email—that are error-prone, insecure and difficult to audit. This project presents **LexManage**, an enterprise-grade, multi-tenant Software-as-a-Service (SaaS) platform that consolidates legal practice management into a single secure web application.

LexManage is built as a decoupled system: a React 19 single-page application communicates with a NestJS REST API backed by PostgreSQL through the Prisma ORM. Strict tenant isolation is enforced at the application layer using Node.js *AsyncLocalStorage* together with a Prisma client extension, guaranteeing that one firm can never access another firm's data. The platform provides role-based access control, JWT authentication with refresh tokens, an immutable audit trail, real-time notifications over Socket.io, secure document storage on MinIO via presigned URLs, and an AI assistant (**LexAssist**) that uses Retrieval-Augmented Generation (RAG)—implemented as n8n workflows with Cohere embeddings, a Pinecone vector database and a DeepSeek language model—to answer questions over case files with verifiable citations.

The system was developed following an iterative, Agile methodology and validated through unit, integration and system testing. The result is a responsive, bilingual (English/French) platform that improves the efficiency, security and traceability of day-to-day legal operations. This report documents the problem analysis, requirements, design, implementation and testing of the platform.

Keywords: Legal Tech, SaaS, Multi-tenancy, NestJS, React, Prisma, RBAC, Retrieval-Augmented Generation, Vector Database, Audit Trail.

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
ALS	AsyncLocalStorage
API	Application Programming Interface
CRM	Client Relationship Management
CRUD	Create, Read, Update, Delete
CI/CD	Continuous Integration / Continuous Delivery
DFD	Data Flow Diagram
DMS	Document Management System
DTO	Data Transfer Object
ERD	Entity-Relationship Diagram
HTTP	HyperText Transfer Protocol
JWT	JSON Web Token
LLM	Large Language Model
MFA	Multi-Factor Authentication
ORM	Object-Relational Mapping
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
REST	Representational State Transfer
SaaS	Software as a Service
SPA	Single-Page Application
SQL	Structured Query Language
UI/UX	User Interface / User Experience
WSS	WebSocket Secure

Figure 3.1: Context (Level-0) Data Flow Diagram of LexManage	17
Figure 3.2: Use Case Diagram of the LexManage platform	17
Figure 3.3: Activity diagram — document ingestion and AI indexing (n8n)	18
Figure 3.4: Activity diagram — creating a new case	19
Figure 3.5: UML class diagram of the LexManage domain model	22
Figure 3.6: Sequence diagram — user authentication (JWT)	23
Figure 3.7: Sequence diagram of the LexAssist Retrieval-Augmented Generation query	23
Figure 3.8: Three-tier system architecture of LexManage (with the n8n AI tier)	24
Figure 3.9: Authentication (login) screen	25
Figure 3.10: Firm onboarding — invitation-based ‘Join the firm’ flow	25
Figure 3.11: Entity-Relationship diagram of the core data model	26
Figure 4.1: Main application shell: navigation, global header and dashboard area	29
Figure 4.2: Creating a case — the ‘Open New Case’ dialog	30
Figure 4.3: Adding a client — the ‘Add a client’ dialog	31
Figure 4.4: Calendar and deadline management view	32
Figure 4.5: Adding an event — the ‘New deadline’ dialog	32
Figure 4.6: Uploading a document — categorisation, access control and drop zone	33
Figure 4.7: Firm management panel (team, invitations and roles)	34
Figure 4.8: Sending a notification — urgency selection step	35
Figure 4.9: n8n document-ingestion & indexing workflow	35
Figure 4.10: n8n Retrieval-Augmented Generation (query) workflow	36
Figure 4.11: LexAssist AI assistant (secure RAG mode) with guided prompts	37
Figure 4.12: User profile and account-security view	38
Figure 4.13: Application settings: language and appearance	38

ATTESTATION	2
DEDICATION	3
ACKNOWLEDGEMENTS	4
ABSTRACT	5
LIST OF ABBREVIATIONS	6
CHAPTER I: GENERAL INTRODUCTION	10
1.1 Introduction	10
1.2 Background of the Study	10
1.3 Statement of the Problem	10
1.4 Objectives of the Study	11
1.4.1 General Objective	11
1.4.2 Specific Objectives	11
1.5 Significance of the Study	11
1.6 Scope of the Study	11
1.7 Definition of Working Terms	11
1.8 Organisation of the Study	12
CHAPTER II: LITERATURE REVIEW	13
2.1 Introduction	13
2.2 Review of Existing Systems	13
2.2.1 Clio	13
2.2.2 MyCase	13
2.2.3 PracticePanther	13
2.2.4 Generic office tools (spreadsheets, shared drives, email)	13
2.2.5 Document management systems (e.g. NetDocuments, iManage)	13
2.2.6 Summary of the gap	14
2.3 Proposed Solution	14
CHAPTER III: MATERIALS AND METHODS USED	15
3.1 Introduction	15
3.2 Methods Used	15
3.2.1 Development Methodology Used	15
3.2.2 System Analysis	15
3.3.2 System Design	20
CHAPTER IV: IMPLEMENTATION, RESULTS AND TESTING	28
4.1 Introduction	28
4.2 Implementation	28
4.2.1 Multi-tenant Engine and Security	28

4.2.2 Dashboard and Case Management	28
4.2.3 Client Directory (CRM)	30
4.2.4 Calendar, Deadlines and Documents	31
4.2.5 Firm Management, RBAC and Notifications	33
4.2.6 LexAssist AI — RAG Workflows with n8n	35
4.2.7 Account Security and Application Settings	37
4.3 Testing	39
4.3.1 Unit Testing	39
4.3.2 Integration Testing	39
4.3.3 System Testing	39
CHAPTER V: CONCLUSIONS AND RECOMMENDATIONS	41
5.1 Conclusions	41
5.2 Recommendations	41
5.3 Perspectives for Further Study	41
REFERENCES	42
APPENDICES	43

CHAPTER I

GENERAL INTRODUCTION

1.1 Introduction

The legal profession is, by its very nature, information-intensive. A single case may involve dozens of clients, hearings, statutory deadlines and hundreds of pages of confidential documents that must be stored, retrieved and acted upon with precision. As caseloads grow, the tools many firms use to coordinate this work—spreadsheets, shared folders and email threads—struggle to keep pace. This chapter introduces **LexManage**, a web-based legal practice management platform designed to address these challenges, and sets out the background, problem, objectives and scope of the project.

1.2 Background of the Study

Digital transformation has reshaped most professional services, yet legal practice management in many jurisdictions remains comparatively under-digitised. Where dedicated software exists it is often expensive, installed on a single machine, or hosted abroad with limited regard for local data-protection expectations. Meanwhile, advances in cloud computing have popularised the **Software-as-a-Service (SaaS)** delivery model, in which a single, centrally maintained application securely serves many independent organisations (tenants) over the internet.

In parallel, the maturation of **Large Language Models (LLMs)** has created an opportunity to augment knowledge work. When combined with **Retrieval-Augmented Generation (RAG)**—a technique that grounds an LLM’s answers in an organisation’s own documents—these models can summarise lengthy files, draft routine clauses and answer questions with traceable citations. LexManage was conceived to bring both of these advances, secure multi-tenant SaaS and document-grounded AI, to the day-to-day operations of a law firm.

1.3 Statement of the Problem

Law firms that rely on disconnected, manual tools face a set of recurring and costly problems:

- **Fragmented data.** Client details, case files and documents live in separate places, making a complete, reliable view of a matter difficult to obtain.
- **Weak security and confidentiality.** Shared drives and email offer little access control, putting privileged information at risk and complicating compliance.
- **Missed deadlines.** Hearings and statutory time-limits tracked in personal calendars are easily overlooked, with serious professional consequences.
- **No traceability.** Without an audit trail it is hard to know who changed what and when, which undermines accountability.
- **Slow document review.** Manually reading and summarising voluminous case files consumes billable hours that could be spent on higher-value legal reasoning.

The core problem this project addresses is therefore the **absence of a single, secure and intelligent system** that unifies client, case, calendar and document management for a law firm while guaranteeing strict data isolation between firms.

1.4 Objectives of the Study

1.4.1 General Objective

To design and implement a secure, multi-tenant SaaS platform that centralises and streamlines the management of clients, cases, hearings and documents for law firms, enhanced with an AI assistant for document analysis.

1.4.2 Specific Objectives

1. To analyse the operational needs of a typical law firm and derive functional and non-functional requirements.
2. To design a multi-tenant architecture that enforces strict data isolation between firms.
3. To implement secure authentication and role-based access control (admin, lawyer, assistant).
4. To build modules for client (CRM), case, calendar/deadline and document management.
5. To integrate a Retrieval-Augmented Generation assistant for summarising and querying case documents.
6. To provide real-time notifications and an immutable audit trail for accountability.
7. To validate the platform through unit, integration and system testing.

1.5 Significance of the Study

For **legal practitioners**, LexManage reduces administrative overhead, lowers the risk of missed deadlines and accelerates document review, freeing time for substantive legal work. For **firm administrators**, it provides governance through RBAC, audit logging and centralised configuration. For the **discipline of software engineering**, the project is a concrete case study in building a secure multi-tenant SaaS system and in applying RAG to a regulated, confidentiality-sensitive domain.

1.6 Scope of the Study

The project covers the design and implementation of a working web platform comprising: firm onboarding and team invitations; authentication and RBAC; client, case, calendar and document management; real-time notifications; an audit trail; and an AI document-analysis assistant. The work focuses on the software system itself. Native mobile applications, online payment processing, integration with external court e-filing systems and large-scale production deployment/operations are considered out of scope, although the architecture is designed to accommodate such extensions in future.

1.7 Definition of Working Terms

Table 1.1: Definition of key working terms

12

Term	Meaning in this report
Tenant	An isolated organisation (law firm) within the SaaS platform; all of a tenant’s data is private to it.
Multi-tenancy	An architecture where one running application instance securely serves many tenants.
RBAC	Role-Based Access Control—permissions granted according to a user’s role.
JWT	A signed token used to authenticate API requests without server-side session storage.
RAG	Retrieval-Augmented Generation—grounding an LLM’s answers in retrieved documents.
Vector database	A store of numerical ‘embeddings’ enabling semantic similarity search (Pinecone).
Presigned URL	A temporary, signed link granting time-limited access to a stored file.
Audit trail	An immutable log of who performed which action on which record and when.

1.8 Organisation of the Study

The remainder of this report is organised as follows. **Chapter II** reviews existing systems and positions the proposed solution. **Chapter III** presents the materials and methods used, covering the development methodology, system analysis (requirements and diagrams) and system design (architecture, data model and user-interface design). **Chapter IV** describes the implementation, results and testing of the platform. **Chapter V** concludes the report and offers recommendations and perspectives for further work.

CHAPTER II

LITERATURE REVIEW

2.1 Introduction

This chapter surveys existing approaches to legal practice management and the technologies that underpin them, in order to identify gaps that LexManage seeks to fill. It examines representative commercial systems, the general-purpose tools firms improvise with, and the technical foundations of multi-tenancy and Retrieval-Augmented Generation, before stating the proposed solution.

2.2 Review of Existing Systems

A range of solutions are used, formally or informally, to manage legal work. The most relevant categories and representative examples are reviewed below.

2.2.1 Clio

Clio is a mature, cloud-based legal practice management suite offering case management, billing and client intake. It is feature-rich and widely adopted, but it is a commercial product priced per user per month, hosted in foreign data centres, and offers limited document-grounded AI; its cost and data-residency model can be unsuitable for smaller or locally focused firms.

2.2.2 MyCase

MyCase provides case management, client communication and invoicing with a friendly interface. Like Clio it is subscription-based and closed-source, and its automation focuses on workflow templates rather than on AI-assisted analysis of the firm's own documents.

2.2.3 PracticePanther

PracticePanther emphasises automation and ease of use for small firms. It covers the core management functions well but, again, is a proprietary hosted service with limited transparency over how and where data is stored and no open, document-grounded AI capability.

2.2.4 Generic office tools (spreadsheets, shared drives, email)

Many firms manage matters with general-purpose tools. These are inexpensive and familiar but provide no access control beyond folder permissions, no audit trail, no structured deadline tracking and no protection against data being mixed between unrelated matters or clients.

2.2.5 Document management systems (e.g. NetDocuments, iManage)

Dedicated legal DMS products excel at secure document storage and versioning but are typically expensive, focus narrowly on documents rather than the full practice workflow, and are not designed around modern document-grounded AI assistants.¹⁴

2.2.6 Summary of the gap

The review shows that existing options force a trade-off: powerful commercial suites are costly, proprietary and host data abroad with little AI grounding, while improvised tools are cheap but insecure and fragmented. None combine, in a single accessible system: strict multi-tenant isolation, role-based governance, an audit trail, real-time alerts and an AI assistant that answers questions over the firm’s own documents with citations.

***Table 2.1:** Comparison of existing systems with the proposed solution*

Capability	Commercial suites	Generic tools	Legal DMS	LexManage
Multi-tenant SaaS	Yes	No	Partial	Yes
RBAC & audit trail	Partial	No	Partial	Yes
Case + calendar + CRM	Yes	Manual	No	Yes
Secure document store	Yes	No	Yes	Yes
Document-grounded AI (RAG)	Limited	No	No	Yes
Real-time notifications	Partial	No	No	Yes

2.3 Proposed Solution

LexManage is proposed as an integrated, multi-tenant SaaS platform that unifies the strengths of the reviewed categories while closing their gaps. It delivers full practice management (clients, cases, calendar, documents) with enterprise-grade security (JWT authentication, RBAC, strict tenant isolation, audit logging and presigned-URL document access), real-time collaboration via Socket.io, and a built-in AI assistant—**LexAssist**—that applies Retrieval-Augmented Generation over the firm’s documents to provide summaries, drafts and grounded answers with citations. The design, methods and implementation that realise this solution are detailed in the following chapters.

CHAPTER III

MATERIALS AND METHODS USED

3.1 Introduction

This chapter describes how the LexManage platform was engineered. It first presents the development methodology, then the system analysis—requirements, data-flow, use-case and activity models, cost and schedule—and finally the system design, covering the tools used, architecture, class and sequence models, user-interface design, the entity-relationship model and the data dictionary.

3.2 Methods Used

3.2.1 Development Methodology Used

An **iterative and incremental (Agile)** methodology was adopted. Rather than attempting to specify the entire system up front, development proceeded in short cycles, each delivering a working slice of functionality (for example, authentication, then case management, then the AI assistant). Every iteration comprised requirements refinement, design, implementation and testing, allowing feedback to be incorporated continuously and risk to be reduced early. This approach suited a single-developer project of evolving scope and aligned naturally with the modular structure of the NestJS backend and the component-based React frontend.

3.2.2 System Analysis

System analysis translates the problem and objectives of Chapter I into a precise specification of what the platform must do and the constraints under which it must operate.

External Interface Requirements

- **User interface:** a responsive web SPA accessible from any modern browser, with light/dark themes and English/French localisation.
- **Software interfaces:** a RESTful HTTP/JSON API; a WebSocket (Socket.io) channel for real-time notifications; the MinIO S3 API for object storage; the Pinecone API for vector search; and n8n workflows (DeepSeek and Cohere) for the AI assistant.
- **Hardware interfaces:** none beyond a standard internet-connected client device and a server host; no specialised peripherals are required.
- **Communication interfaces:** HTTPS for all client-server traffic and WSS for the real-time channel; SMTP for outgoing invitation and reminder e-mails.

Features or Modules of the System

- **Identity & Organisation:** firm onboarding, secure token-based invitations, RBAC.
- **Client Management (CRM):** directory of physical and legal-entity clients and their matters.
- **Case Management:** full lifecycle of legal files with status, priority and assignment.
- **Calendar & Deadlines:** hearing and deadline scheduling with automated reminders.

- **Document Management:** categorised, role-restricted document library on secure storage.
- **LexAssist AI:** RAG-based summarisation, drafting and grounded Q&A over documents.
- **Notifications:** real-time and scheduled alerts with urgency levels.
- **Audit & Statistics:** immutable audit trail and dashboard key-performance indicators.

Functional Requirements

Table 3.1: Functional requirements of the system

ID	Requirement
FR-1	The system shall allow an administrator to create a firm and invite collaborators via a secure, expiring token.
FR-2	The system shall authenticate users with email/password and issue JWT access and refresh tokens.
FR-3	The system shall enforce role-based permissions for admin, lawyer and assistant roles.
FR-4	The system shall let authorised users create, read, update and archive clients, cases and documents.
FR-5	The system shall schedule hearings/deadlines and send reminders ahead of due dates.
FR-6	The system shall store documents securely and serve them through time-limited presigned URLs.
FR-7	The system shall answer natural-language questions over a firm’s documents using RAG, returning citations.
FR-8	The system shall deliver real-time notifications and highlight urgent alerts.
FR-9	The system shall record an immutable audit log of create/update actions on key entities.
FR-10	The system shall isolate each firm’s data so it is inaccessible to any other firm.

Non-functional Requirements

Table 3.2: Non-functional requirements of the system

ID	Category	Requirement
NFR-1	Security	All traffic over HTTPS; passwords hashed; secrets held server-side; presigned-URL file access.
NFR-2	Privacy/Isolation	Strict per-tenant data isolation enforced automatically on every query.
NFR-3	Performance	Typical interactive requests served in under ~300 ms; calendar views optimised for O(1) lookups.
NFR-4	Usability	Responsive, accessible UI with bilingual support and clear empty/error states.
NFR-5	Reliability	Graceful error handling and retry; no single failed request crashes the UI.
NFR-6	Maintainability	Modular, layered codebase with DTO validation and centralised error handling.
NFR-7	Scalability	Stateless API enabling horizontal scaling behind a load balancer.

Data Flow Diagrams (DFDs)

The context (level-0) data-flow diagram in Figure 3.1 situates LexManage among its external entities—firm staff, the n8n AI service (DeepSeek, Cohere and Pinecone), the MinIO object store and the e-mail gateway—and shows the principal data exchanged with each.

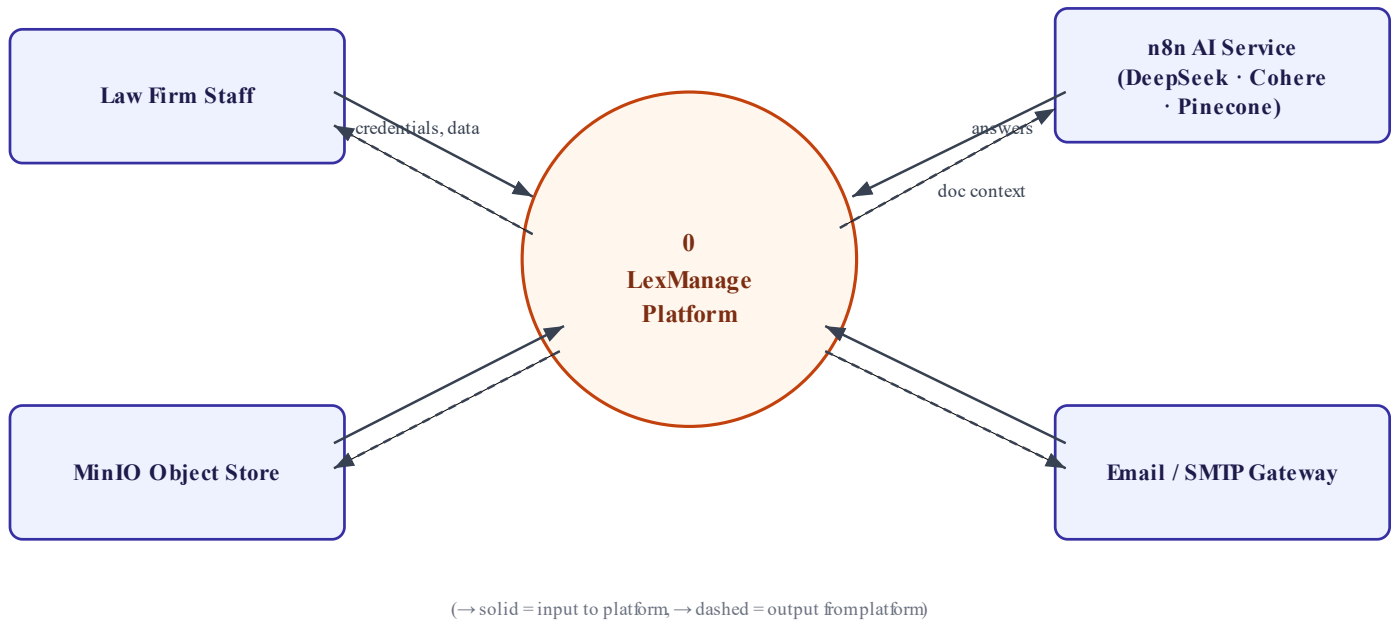


Figure 3.1: Context (Level-0) Data Flow Diagram of LexManage

Use Case Analysis and Diagrams

The principal actors are the **Cabinet Administrator**, the **Lawyer** and the **Assistant/Secretary**. The administrator manages the firm and its members; lawyers manage cases, clients, documents and use the AI assistant; assistants support document and calendar work. Figure 3.2 presents the use-case diagram.

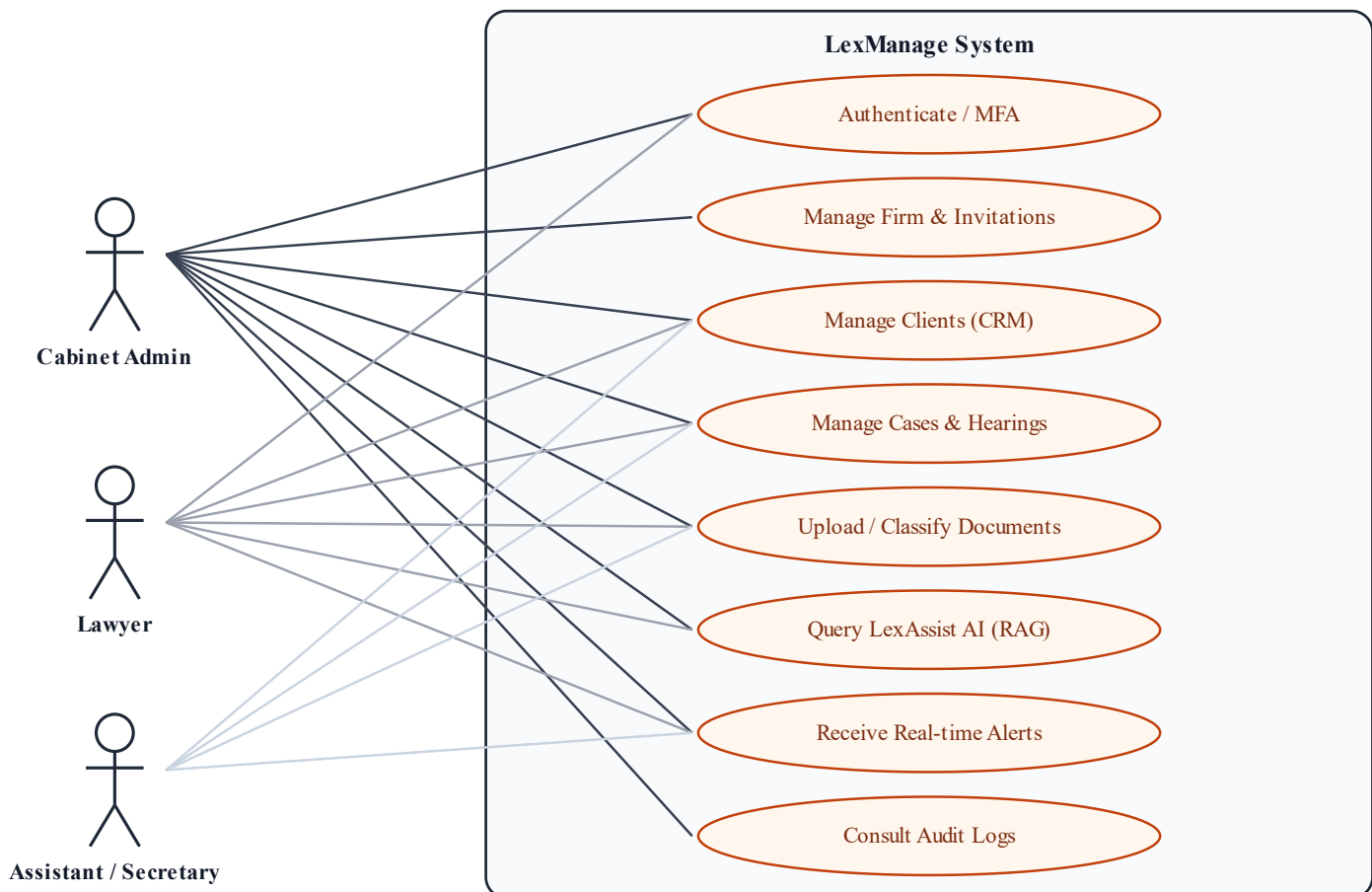


Figure 3.2: Use Case Diagram of the LexManage platform

Two activities capture the platform's most important control flows. The first, Figure 3.3, models **document ingestion and AI indexing**: a file uploaded through the interface is stored in MinIO, after which an n8n workflow extracts its text (branching on file type), splits it into overlapping chunks, generates Cohere embeddings and upserts the resulting vectors into Pinecone before the document is marked as indexed.

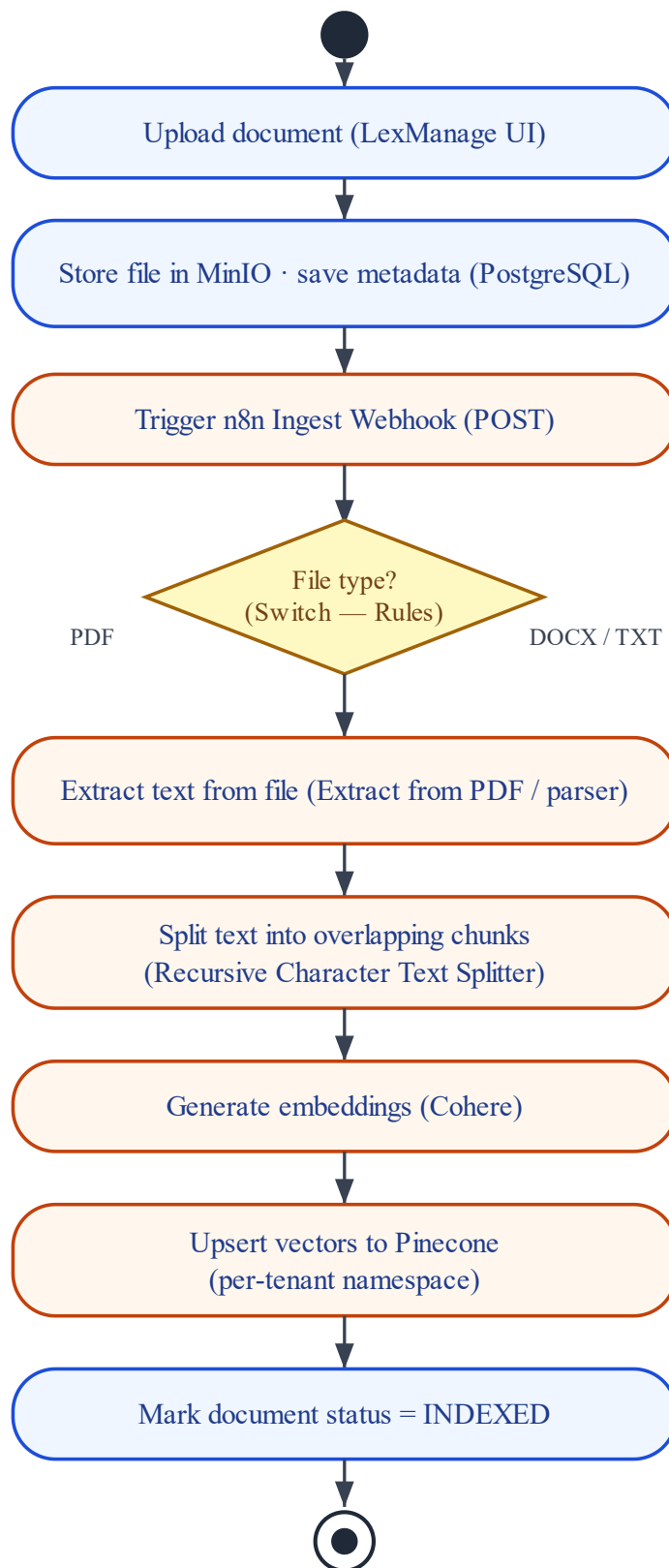


Figure 3.3: Activity diagram — document ingestion and AI indexing (n8n)

The second, Figure 3.4, models **creating a new case**: the user completes the form, the input is validated (looping back on error), the case is persisted within the firm's tenant scope, and an audit entry and a real-time notification are produced in parallel before the

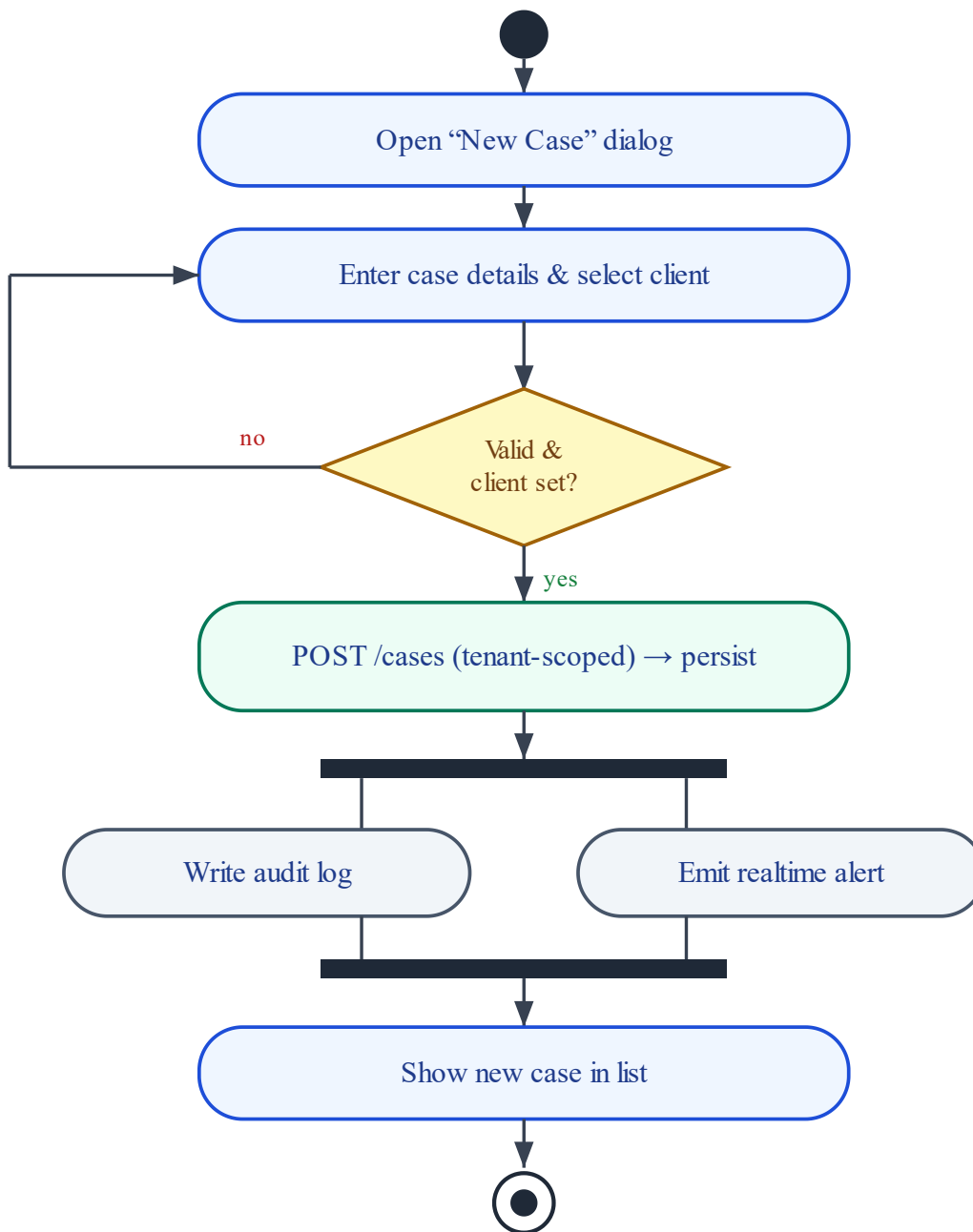


Figure 3.4: Activity diagram — creating a new case

Cost Evaluation

Because the platform is built entirely on free and open-source software and free service tiers, the development cost is dominated by human effort. An indicative evaluation is given below.

Table 3.3: Indicative cost evaluation

Item	Description	Estimated cost
Development effort	~4 months, single developer (opportunity cost)	Time-based
Software tools	NestJS, React, PostgreSQL, MinIO, n8n, VS Code (all open-source)	0
AI services	DeepSeek (chat), Cohere (embeddings/rerank) and Pinecone (vector store) on free/low-cost tiers	Low / metered
Hosting (dev)	Local Docker environment	0
Hosting (production, indicative)	Cloud VPS + managed PostgreSQL	~\$20–50 / month

Project Schedule

Table 3.4: Project schedule by phase

Phase	Activities	Duration
1. Analysis	Requirements gathering, literature review	Weeks 1–2
2. Design	Architecture, data model, UI design	Weeks 3–4
3. Core implementation	Auth, RBAC, multi-tenancy, CRUD modules	Weeks 5–9
4. AI & realtime	RAG assistant, notifications, audit trail	Weeks 10–12
5. Testing & hardening	Unit/integration/system tests, fixes	Weeks 13–14
6. Documentation	Report writing and finalisation	Weeks 15–16

3.3.2 System Design

The design realises the requirements as a decoupled, layered system. This section presents the tools used, hardware and software requirements, the acquisition strategy, the class, sequence and architecture models, the user-interface design, and finally the entity-relationship model and data dictionary.

Tools and Materials Used

The technology stack was selected for type-safety, modularity and a strong open-source ecosystem.

1. Hardware Requirements

Table 3.5: Hardware requirements

Environment	Minimum specification
Developer / server host	Quad-core CPU, 8 GB RAM (16 GB recommended), 20 GB free disk.
Client device	Any modern device capable of running a current web browser.

2. Software Requirements

Table 3.6: *Software tools and technologies used*

21

Layer	Technology	Purpose
Frontend	React 19, Vite, Tailwind CSS	Responsive single-page user interface
State / data	Zustand, TanStack React Query	Global state and server-state caching
Backend	NestJS (Node.js, TypeScript)	Modular REST API and business logic
Database	PostgreSQL + Prisma ORM	Relational, tenant-scoped persistence
Object storage	MinIO (S3-compatible)	Secure document storage with presigned URLs
Vector store	Pinecone	Embedding index for semantic retrieval (RAG)
AI orchestration	n8n	Low-code workflows for document ingestion and RAG queries
Chat model	DeepSeek	Grounded answer generation over retrieved context
Embeddings & rerank	Cohere	Document/query embeddings and relevance reranking
Realtime	Socket.io	Live notifications and updates
Auth	JWT (access + refresh)	Stateless authentication
Tooling	Docker, Git, ESLint	Environment, version control, quality

System Acquisition Strategy

A **build** rather than **buy** strategy was chosen. Commercial suites would impose recurring per-user fees, foreign data residency and limited extensibility, none of which fit the academic and local-firm context of this project. Building on open-source components yields full control over data, security and the AI pipeline, and provides a richer learning outcome.

Class Diagram

At the domain level the principal classes mirror the persistence model. Figure 3.5 presents the UML class diagram. *Tenant* composes *User*, *Client*, *Case*, *Document*, *Event/Deadline*, *Notification*, *Invitation* and *AuditLog*, guaranteeing that every object is owned by exactly one firm. A *Case* belongs to a *Client*, is assigned to a *User*, and aggregates *Document* and *Event* objects; each class carries the attributes and key operations used by the application.

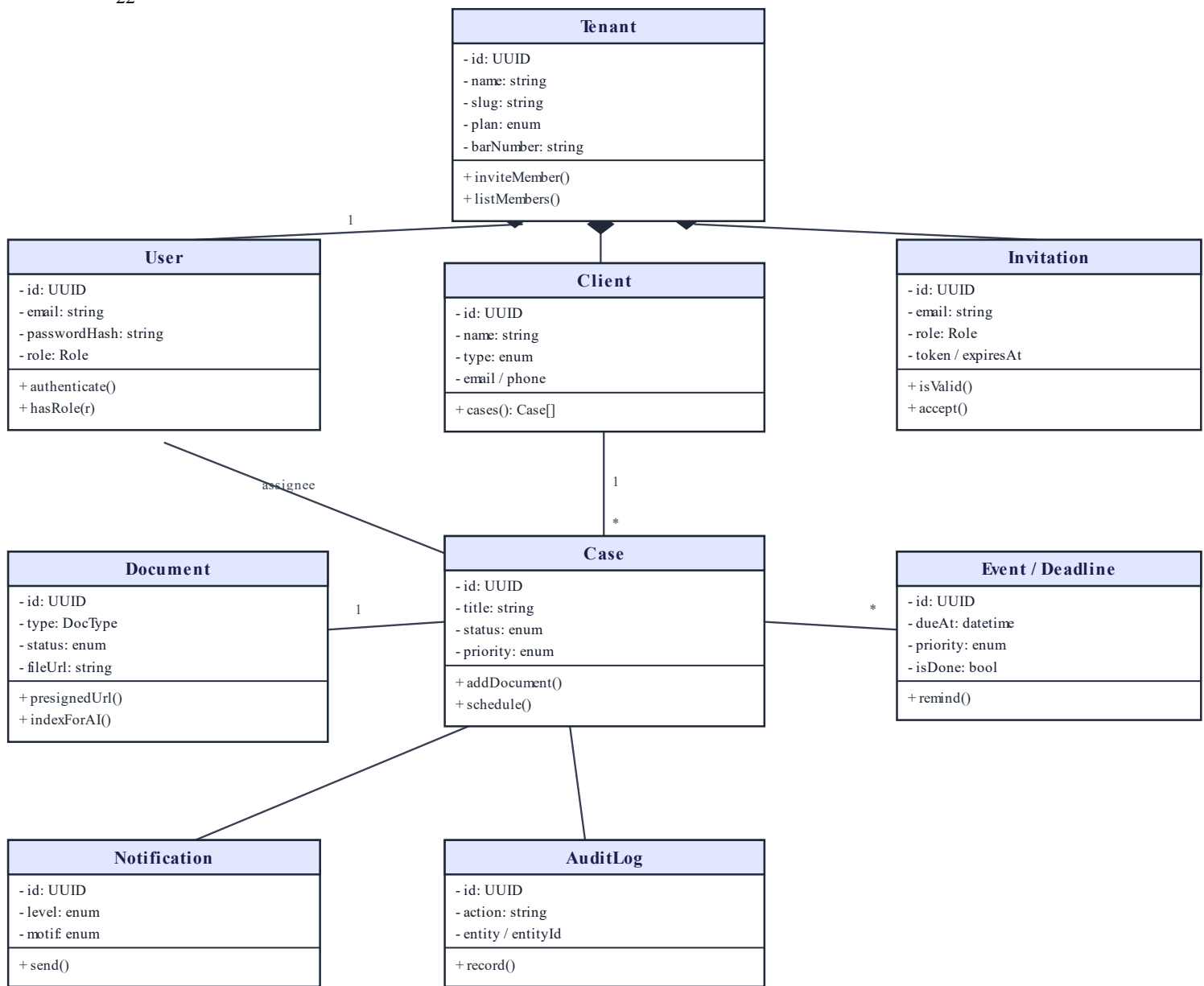


Figure 3.5: UML class diagram of the LexManage domain model

Sequence Diagrams

Two interactions illustrate the dynamic behaviour of the system. Figure 3.6 shows the **authentication** sequence: the user's credentials are verified against the database, the API signs a short-lived JWT access token and a refresh cookie, and the SPA stores the token and redirects to the dashboard.

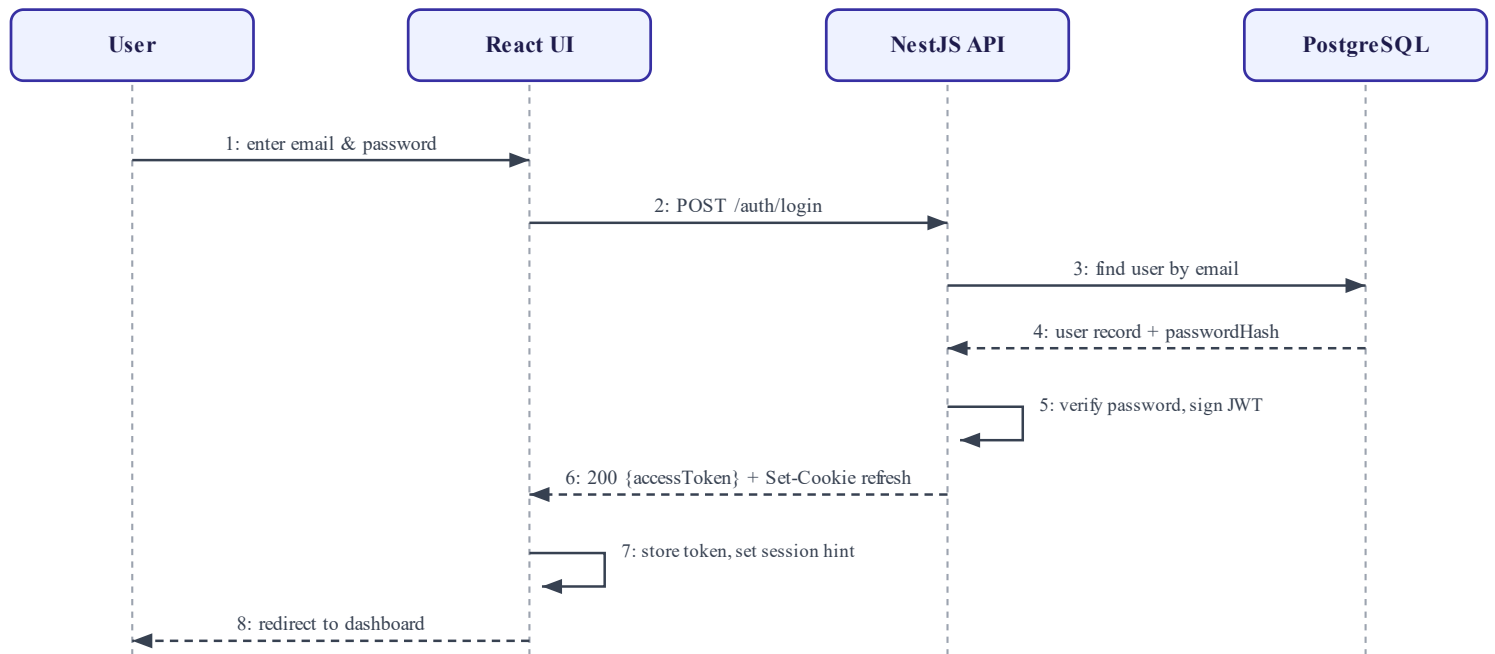


Figure 3.6: Sequence diagram — user authentication (JWT)

Figure 3.7 shows the **LexAssist AI query**, the system’s most distinctive workflow. The request is forwarded by the NestJS API to an n8n workflow whose AI Agent embeds the question with Cohere, performs a reranked similarity search in Pinecone, and passes the retrieved context to the DeepSeek chat model, which returns a grounded answer with citations.

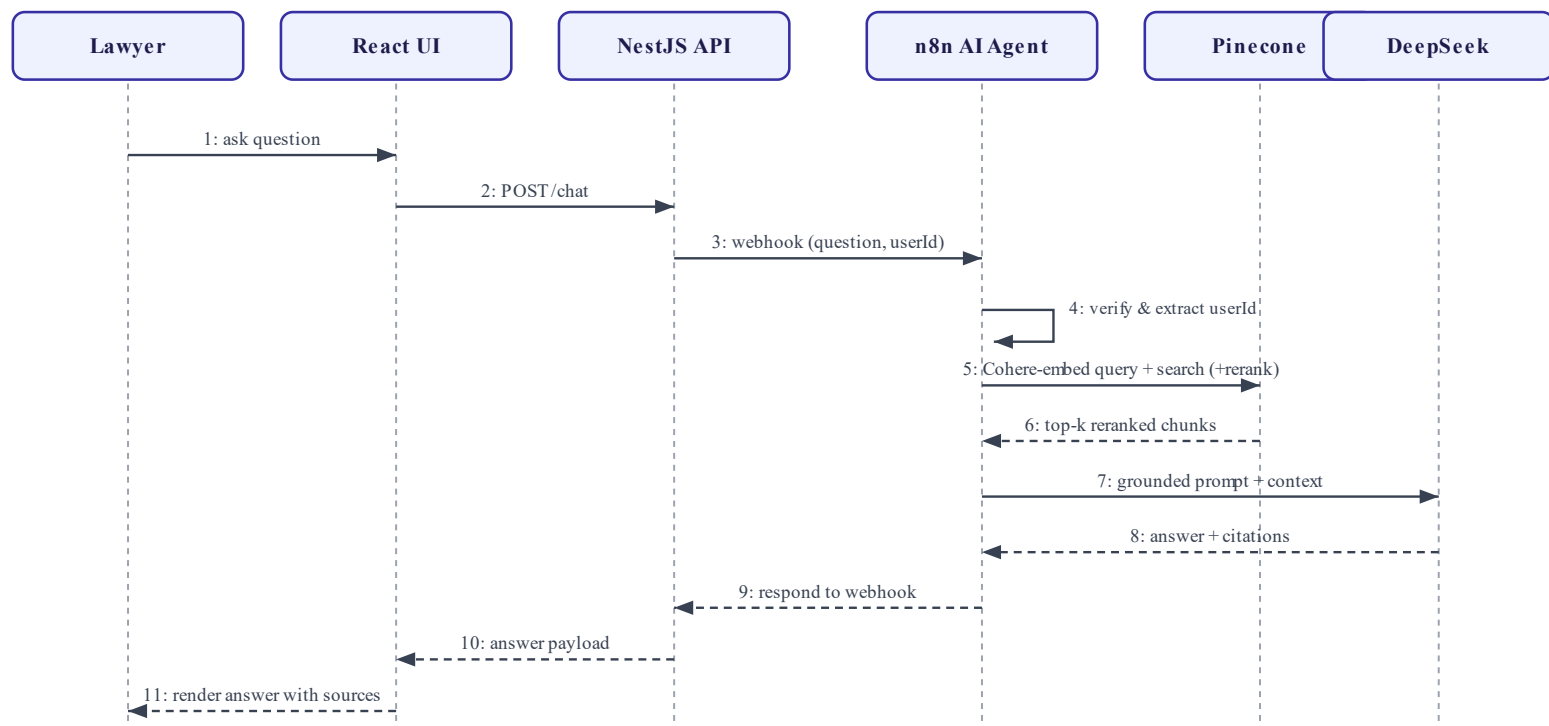


Figure 3.7: Sequence diagram of the LexAssist Retrieval-Augmented Generation query

System Architecture

LexManage follows a classic three-tier architecture, decoupled across a presentation tier (React SPA), an application tier (NestJS API enforcing authentication, RBAC and tenant isolation) and a data tier (PostgreSQL, MinIO and the Pinecone vector store). The AI capability is provided by a separate **AI-automation tier built with n8n**, which the API calls for ingestion and queries and which

in turn uses the DeepSeek chat model and Cohere embeddings/reranker. Tenant isolation is enforced at the application tier using *AsyncLocalStorage*²⁴ to carry the current tenant's identity through each request and a Prisma client extension that automatically scopes every query to that tenant. Figure 3.8 illustrates the architecture.

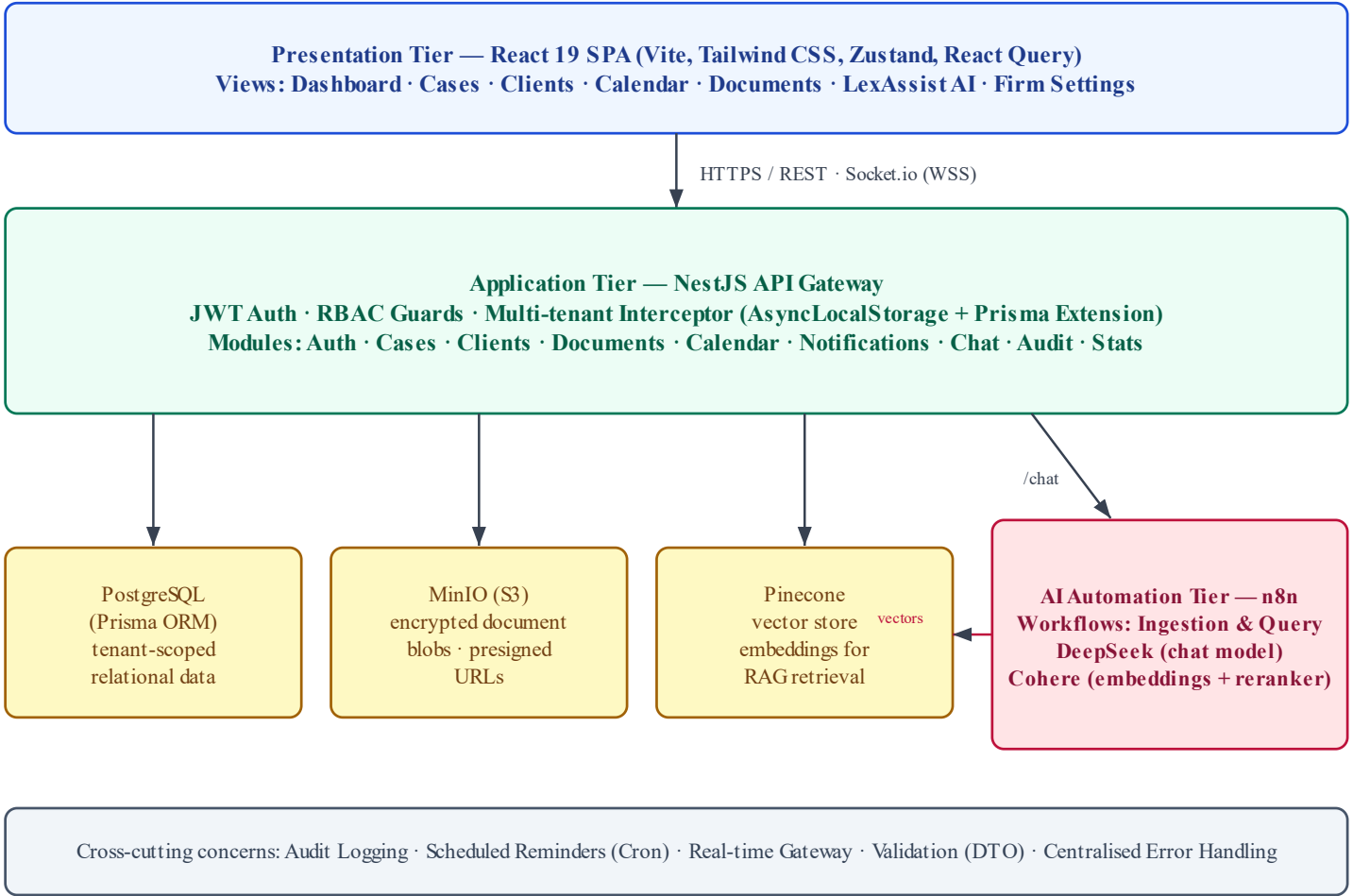


Figure 3.8: Three-tier system architecture of LexManage (with the n8n AI tier)

User Interface Design

The interface follows a clean, professional design language with a persistent navigation sidebar, a global header (search, notifications and the AI entry point) and a content area that renders the active view. Care was taken to provide meaningful empty and error states. Figure 3.9 shows the secure authentication screen and Figure 3.10 the firm-onboarding (invitation) flow.

LEXMANAGE

The reference platform for excellence in law firms.

Manage your cases, automate your billing, and collaborate securely.

Multi-Tenant Isolation
Data strictly partitioned by firm.

© 2026 LEXMANAGE SYSTEMS • SAAS EDITION

Welcome

Log in to your secure workspace.

Work Email

Password

[Forgot password?](#)

[→ Login](#)

Don't have an account yet?

[+ Create a new firm](#)

Figure 3.9: Authentication (login) screen

LEXMANAGE

The reference platform for excellence in law firms.

Manage your cases, automate your billing, and collaborate securely.

Multi-Tenant Isolation
Data strictly partitioned by firm.

© 2026 LEXMANAGE SYSTEMS • SAAS EDITION

[← Back](#)

Join the firm

Step 2: Main Administrator

First Name

Last Name

Work Email

Phone

+237 ...

Password

Password strength: None

- At least 8 characters
- Uppercase letter
- At least 1 number

Confirm Password

[✓ Join now](#)

[Already registered? Login](#)

Figure 3.10: Firm onboarding — invitation-based 'Join the firm' flow

Entity-Relationship (ER) Diagram

The relational data model is centred on the *Tenant* entity, from which all other records descend, guaranteeing isolation by construction. Figure 3.11 presents a simplified ER diagram of the core entities and their relationships.

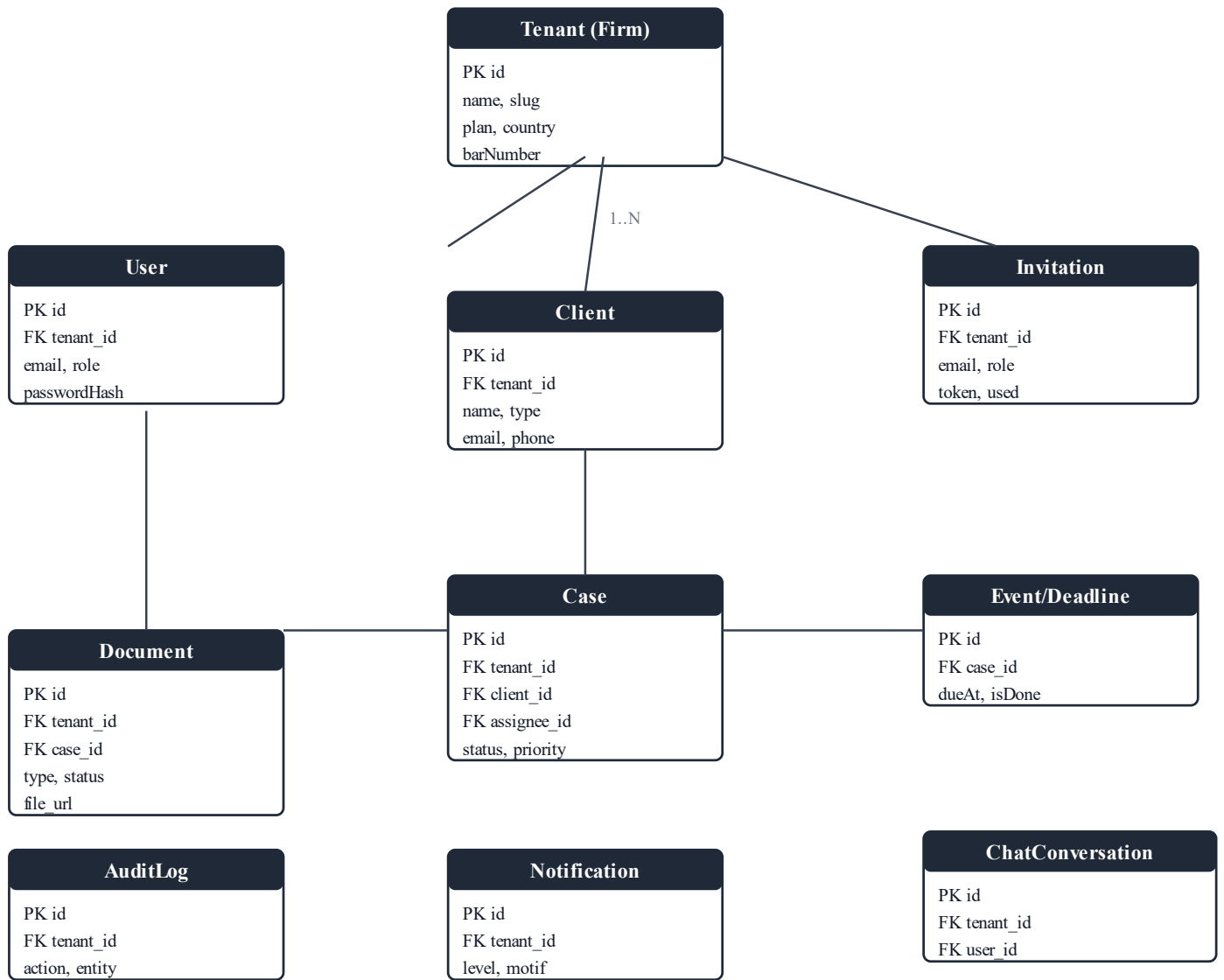


Figure 3.11: Entity-Relationship diagram of the core data model

Data Dictionary

Table 3.7 documents the principal entities and their key attributes.

Table 3.7: Data dictionary of core entities (selected attributes)

27

Entity	Key attributes	Notes
Tenant	id (PK), name, slug, plan, barNumber, country	Root firm/organisation; owns all data.
User	id (PK), tenant_id (FK), email, passwordHash, role	Member of a firm; role drives RBAC.
Client	id (PK), tenant_id (FK), name, type_client, email	CRM entity (physical or legal person).
Case	id (PK), tenant_id (FK), client_id (FK), assignee_id (FK), status, priority	Legal matter/file.
Document	id (PK), tenant_id (FK), case_id (FK), type, status, file_url	Metadata for a file stored in MinIO.
Deadline	id (PK), case_id (FK), dueAt, isDone, priority	Hearing/time-limit; drives reminders.
Notification	id (PK), tenant_id (FK), level, motif, recipient_ids	Real-time/scheduled alert.
AuditLog	id (PK), tenant_id (FK), user_id (FK), action, entity, entityId	Immutable change record.
ChatConversation	id (PK), tenant_id (FK), user_id (FK), messages[]	LexAssist AI conversation history.

CHAPTER IV

IMPLEMENTATION, RESULTS AND TESTING

4.1 Introduction

This chapter reports how the design of Chapter III was implemented, presents the resulting system through its user interface, and describes the testing carried out to validate it.

4.2 Implementation

The system was implemented as two cooperating applications—a React frontend and a NestJS backend—sharing a PostgreSQL database, with MinIO for object storage and a set of n8n workflows (using DeepSeek, Cohere and Pinecone) providing the AI capability, all orchestrated for development with Docker Compose.

4.2.1 Multi-tenant Engine and Security

Every authenticated request carries a JWT from which the user’s tenant is resolved. The tenant identifier is placed into an *AsyncLocalStorage* context that follows the request through the asynchronous call chain. A Prisma client extension then transparently injects a `tenantId` filter into all read and write operations, so that application code cannot accidentally read or write another firm’s data. Authentication issues short-lived access tokens alongside refresh tokens, passwords are hashed, and all sensitive keys (including the AI service credentials) are held server-side only. Role-based guards restrict administrative routes to the *CABINET_ADMIN* and *SUPER_ADMIN* roles.

4.2.2 Dashboard and Case Management

The modules expose validated REST endpoints consumed by the frontend through TanStack React Query, which provides caching, background refresh and optimistic updates. Figure 4.1 shows the main application shell—navigation sidebar, global header and the dashboard, which aggregates key indicators for the firm.

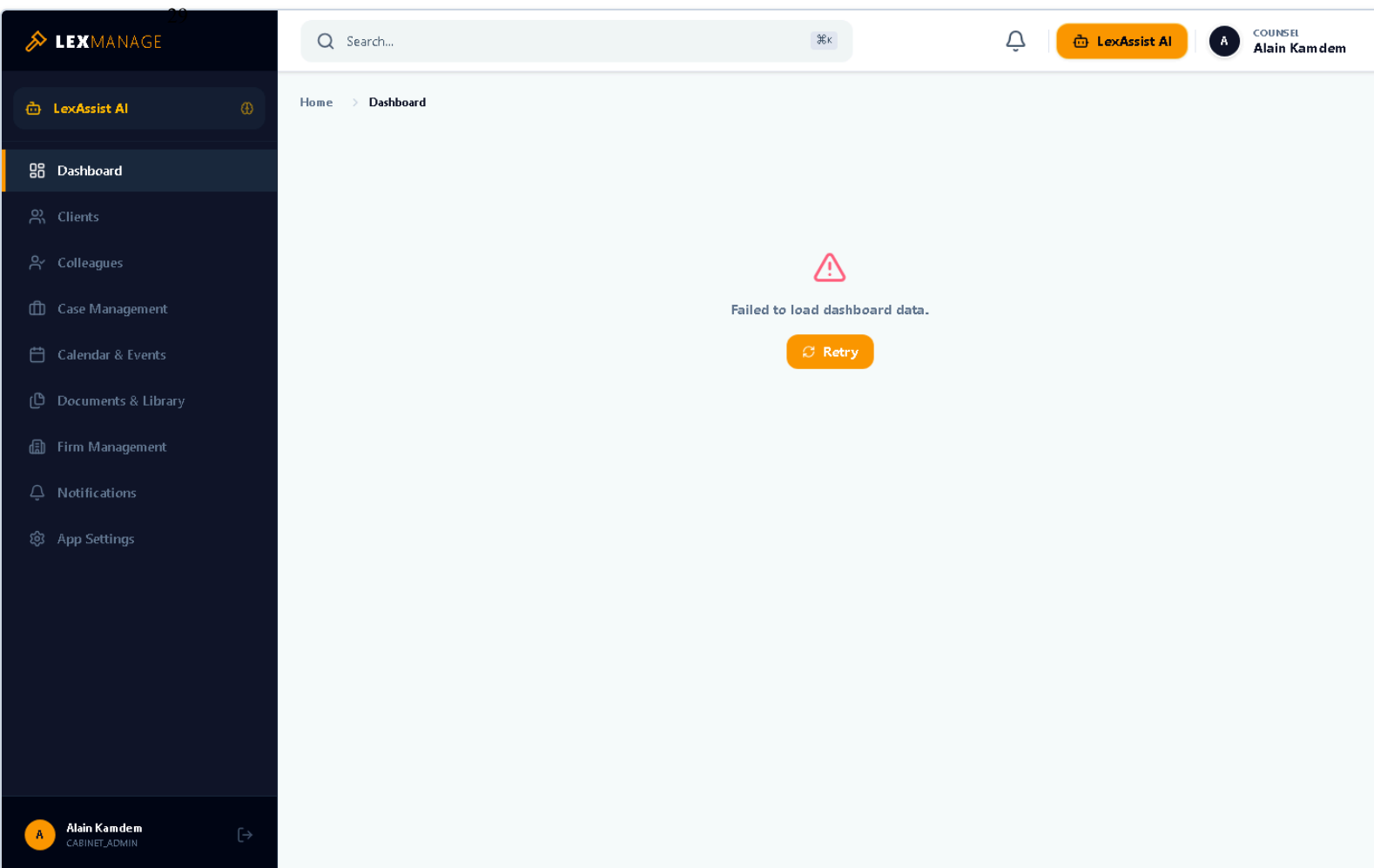


Figure 4.1: Main application shell: navigation, global header and dashboard area

Cases are created from the case-management view through a guided dialog. Figure 4.2 shows the ‘Open New Case’ form, which captures the subject, client, case reference, jurisdiction, initial status and strategic notes, and allows supporting documents to be attached at creation time.

Open New Case
Create a new legal file in the secure repository.

Subject / Case Title *
Globale SARL v. Martin — Breach of Contract

CLIENT *
Select or type client name

Case Number / Reference *
RG-2026-0142

Jurisdiction / Court *
Tribunal de Grande Instance

INITIAL STATUS *
Open

DESCRIPTION / STRATEGIC NOTES
Dispute over non-performance of a commercial supply agreement;
prepare the summons and gather documentary evidence before the first
hearing.

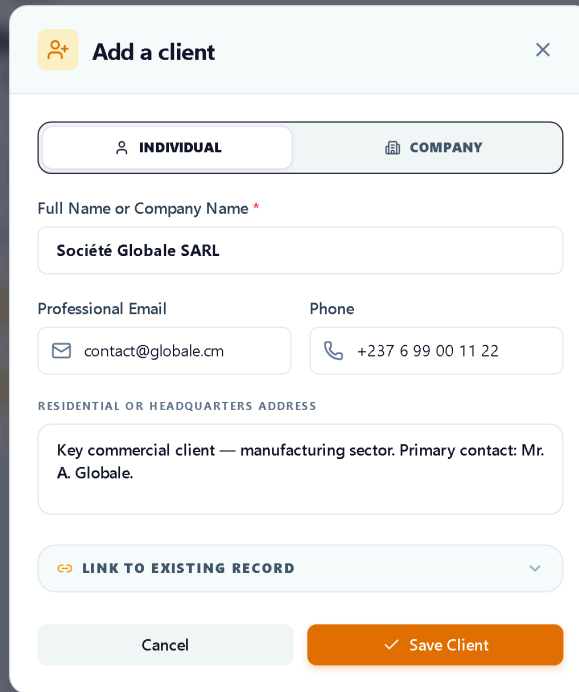
ATTACH DOCUMENTS (OPTIONAL)

Cancel Create Case File

Figure 4.2: Creating a case — the ‘Open New Case’ dialog

4.2.3 Client Directory (CRM)

The CRM module manages the firm’s clients. Figure 4.3 shows the ‘Add a client’ dialog, which distinguishes individual from corporate clients and records the contact details and an optional link to an existing record.



Add a client

☒ INDIVIDUAL ☐ COMPANY

Full Name or Company Name *

Société Globale SARL

Professional Email Phone

contact@globale.cm +237 6 99 00 11 22

RESIDENTIAL OR HEADQUARTERS ADDRESS

Key commercial client — manufacturing sector. Primary contact: Mr. A. Globale.

Figure 4.3: Adding a client — the ‘Add a client’ dialog

4.2.4 Calendar, Deadlines and Documents

The calendar module (Figure 4.4) presents hearings and deadlines in a month grid optimised for fast lookups, with automated reminders generated by scheduled (cron) jobs ahead of due dates. New entries are added through the dialog in Figure 4.5, which records the title, due date, priority and the associated case file.

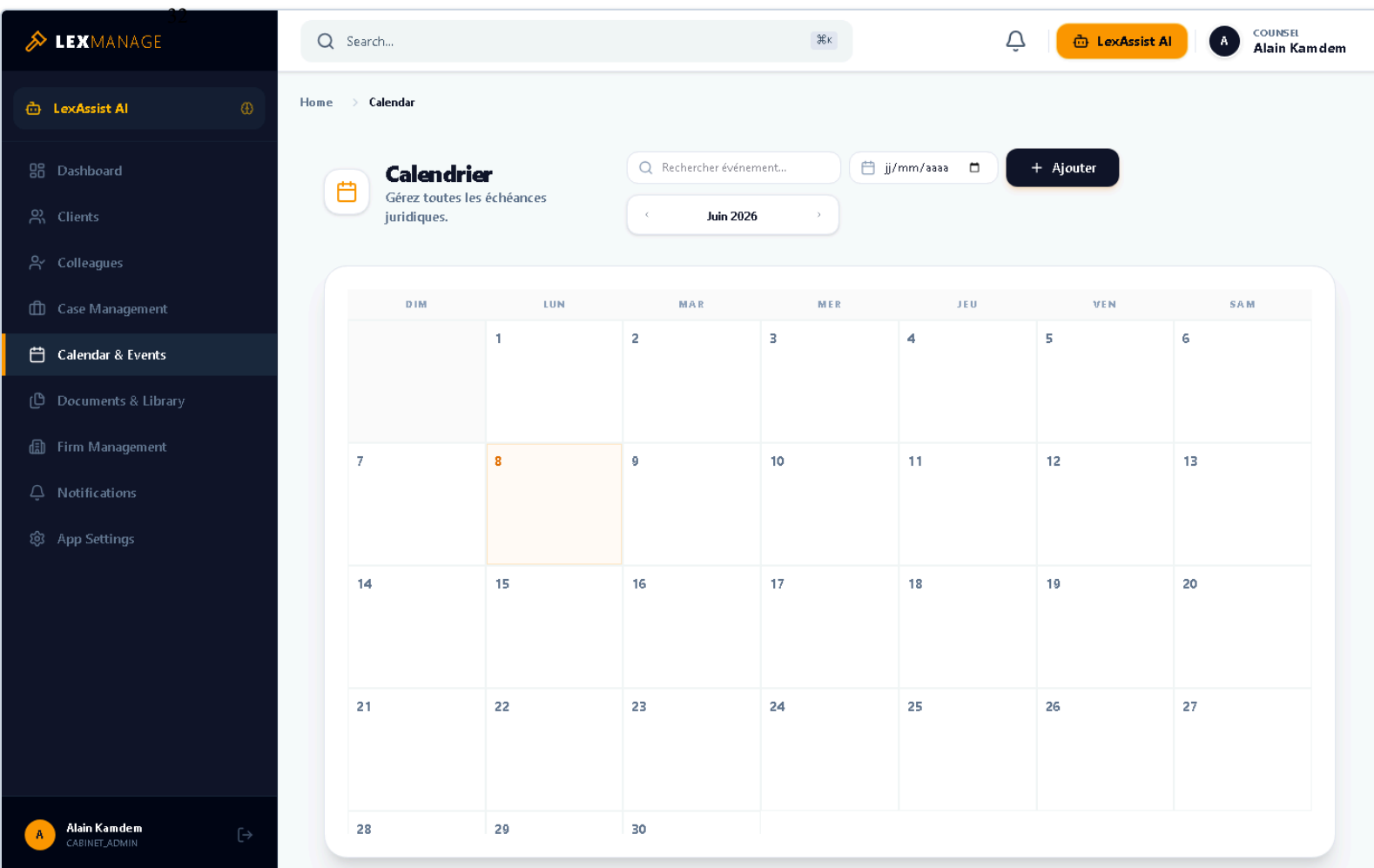


Figure 4.4: Calendar and deadline management view

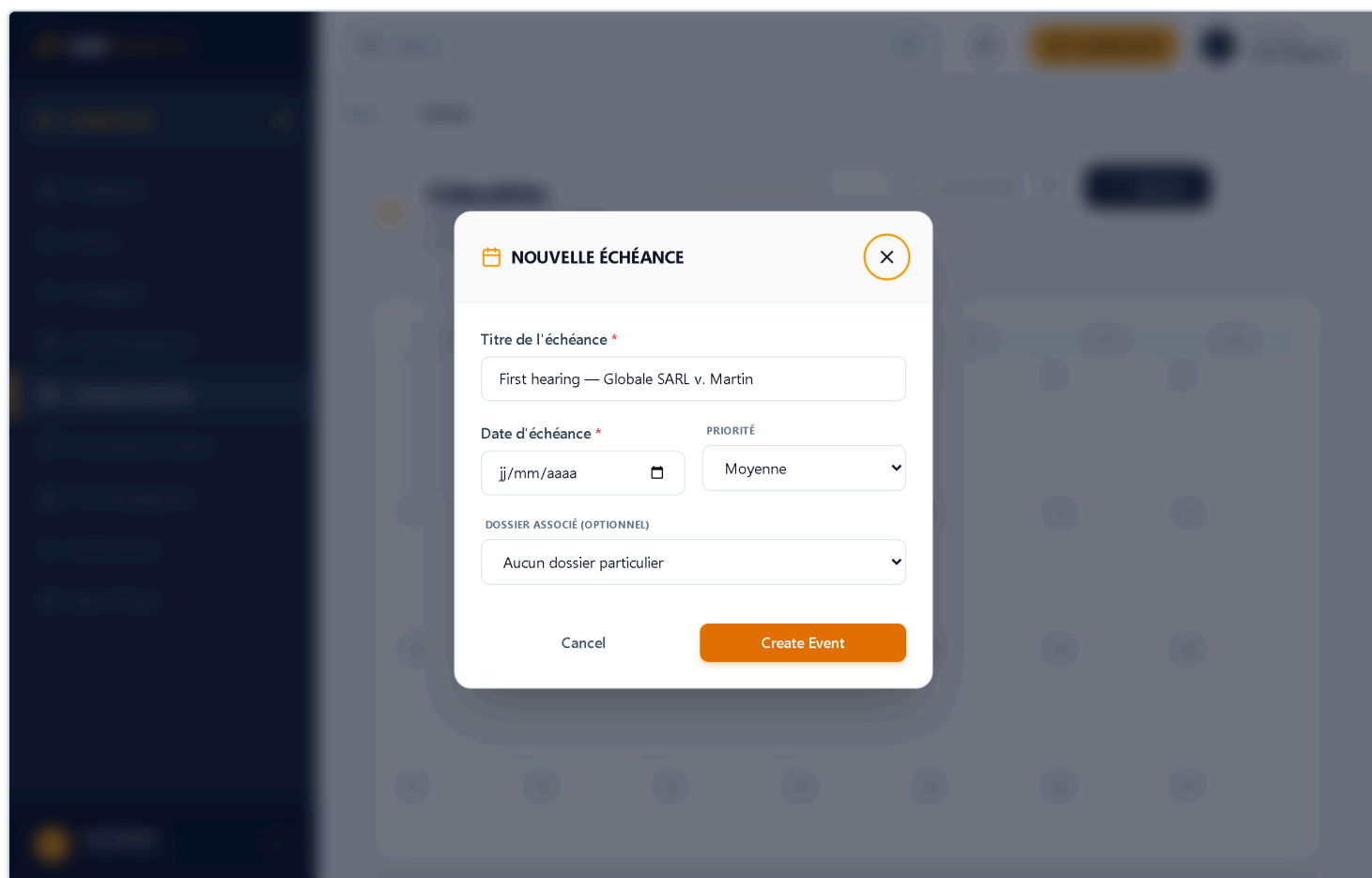


Figure 4.5: Adding an event — the 'New deadline' dialog

Documents are managed in a structured, role-aware library. Figure 4.6 shows the upload panel, where a file is categorised, an access level is chosen (which roles may consult it) and the file is dropped for secure storage in MinIO (PDF, DOCX or TXT).

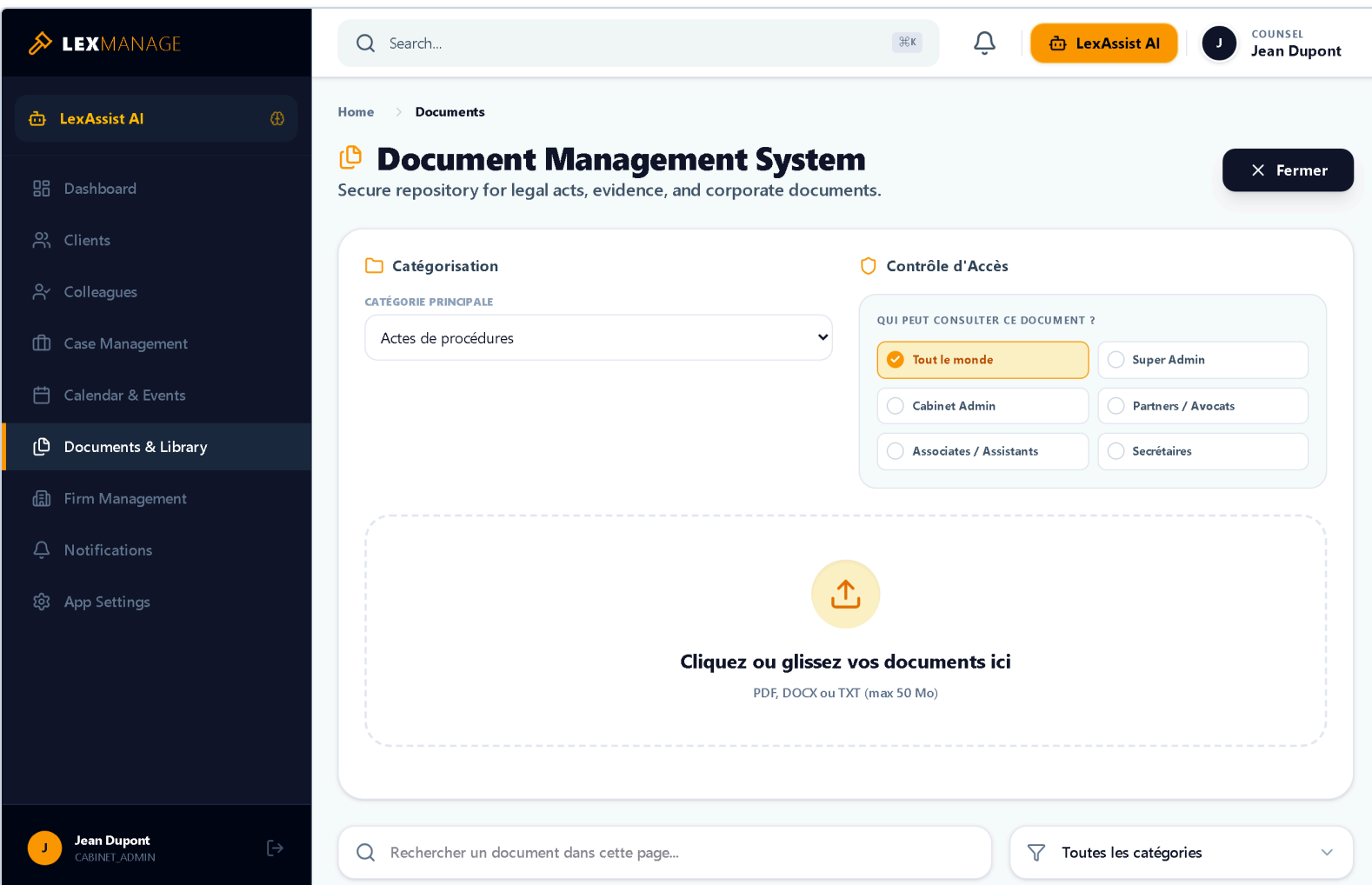


Figure 4.6: Uploading a document — categorisation, access control and drop zone

4.2.5 Firm Management, RBAC and Notifications

Administrators manage their firm from a dedicated panel (Figure 4.7): they can view the team, issue secure, expiring e-mail invitations, assign roles and edit firm information. New members join through the invitation flow shown earlier (Figure 3.10).

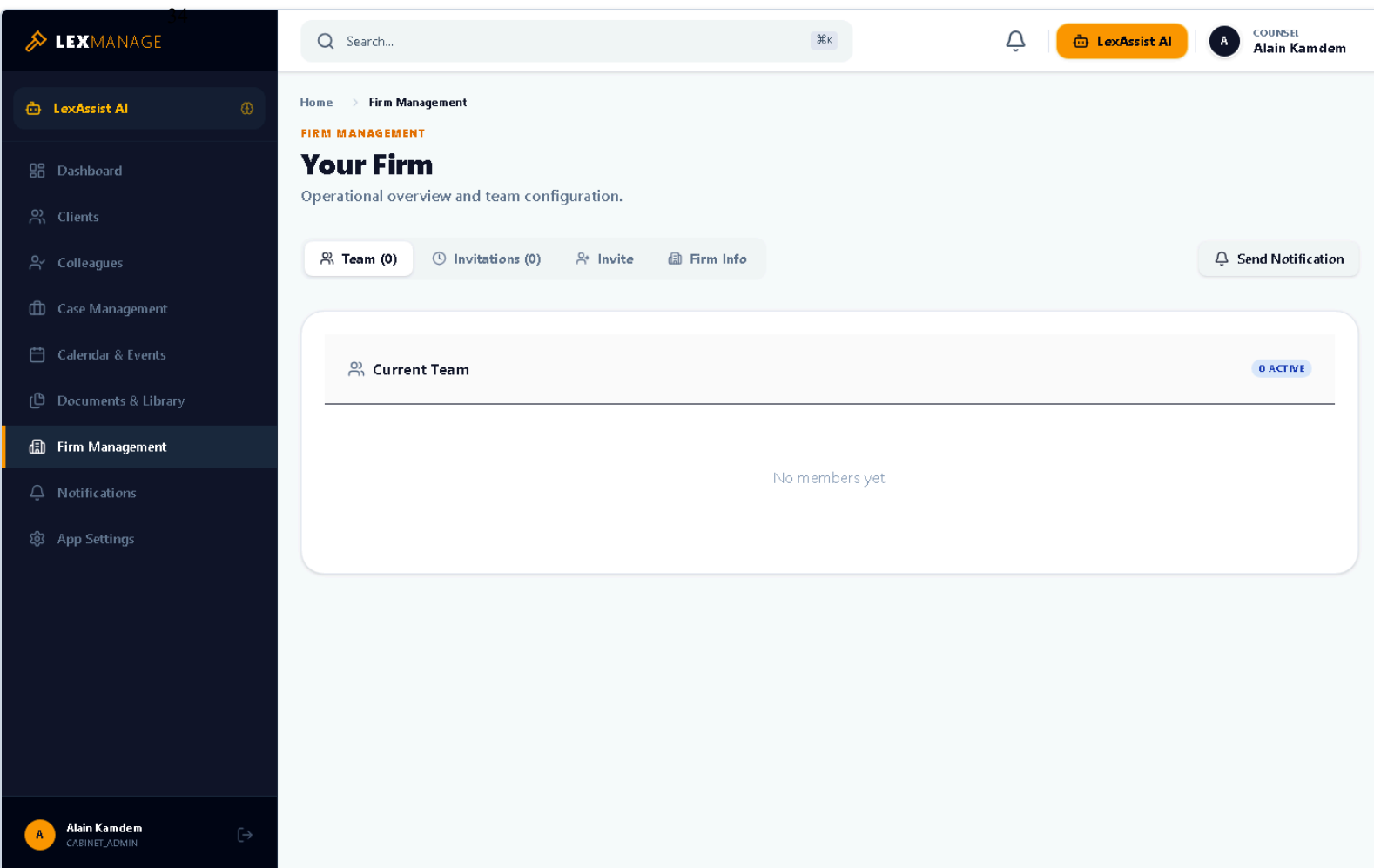


Figure 4.7: Firm management panel (team, invitations and roles)

Administrators can also broadcast alerts to the team. Figure 4.8 shows the first step of the ‘Send Notification’ wizard, where the urgency level (normal, important or urgent) is selected; urgent alerts bypass the standard queue and trigger an immediate pop-up for recipients.

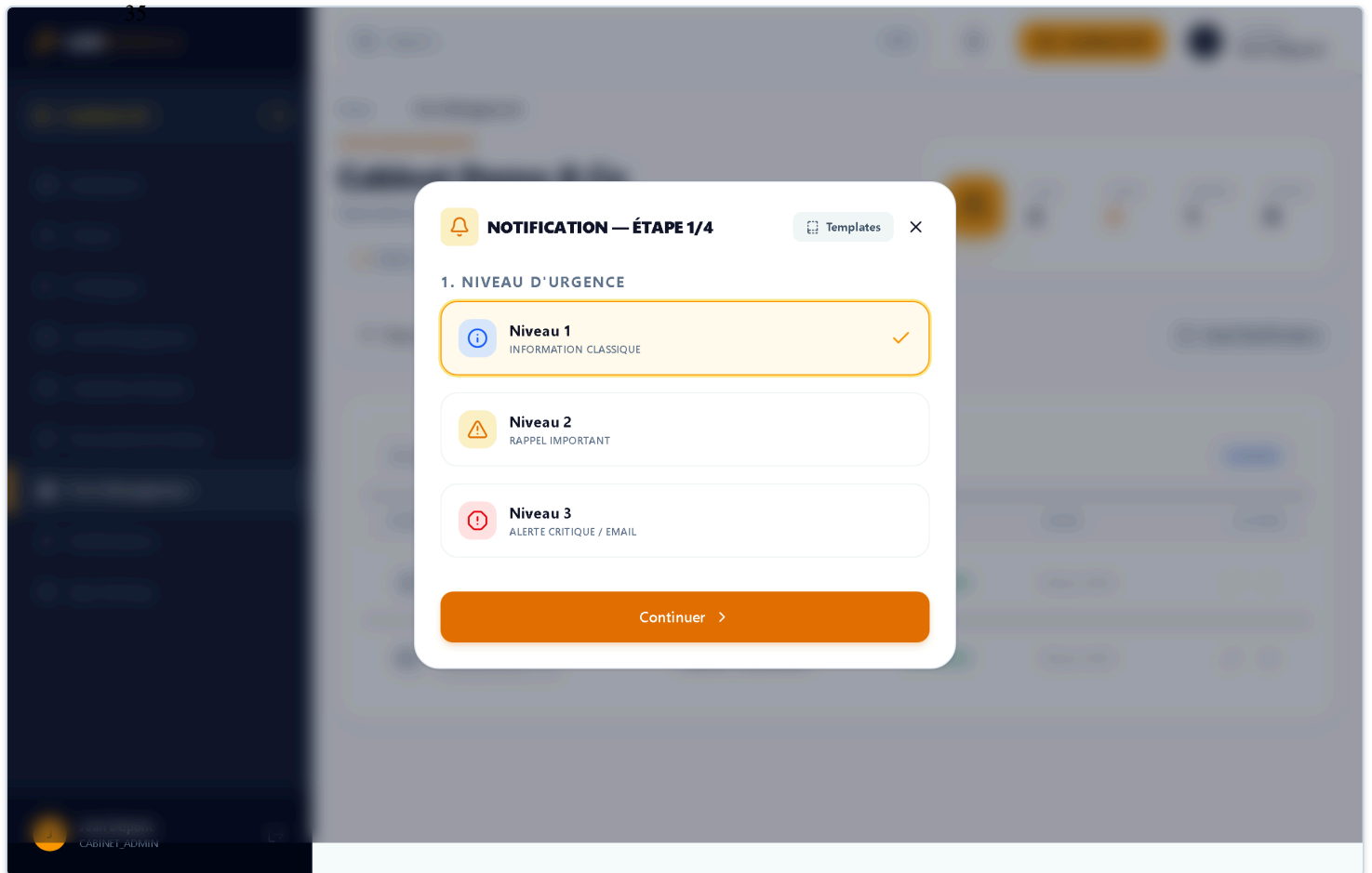


Figure 4.8: Sending a notification — urgency selection step

4.2.6 LexAssist AI — RAG Workflows with n8n

The AI assistant is implemented not inside the backend but as two **n8n** low-code workflows that the NestJS API invokes over webhooks. This keeps AI credentials and orchestration logic outside the client and makes the pipeline easy to inspect and modify. The first workflow ingests and indexes documents; the second answers questions using Retrieval-Augmented Generation.

Figure 4.9 shows the **ingestion workflow**. When a document is uploaded, the workflow extracts its text, splits it into overlapping chunks, embeds them with Cohere and stores the vectors in Pinecone. Table 4.1 explains the role of each node.

[Figure 4.9]

Screenshot to be inserted: figures/n8n-ingest.png

Figure 4.9: n8n document-ingestion & indexing workflow

Table 4.1: Role of each node in the n8n ingestion workflow

Node	Role in the workflow
Ingest Webhook (POST)	Entry point; receives the upload event (document id, tenant, file reference) from the NestJS backend.
Extract / Edit fields	Normalises the payload—document id, tenant namespace, file URL and MIME type.
Switch (Format — Rules)	Routes the item by file type (PDF / DOCX / TXT) to the correct extractor.
Extract from File	Reads the binary file and extracts its raw text content (e.g. Extract from PDF).
Recursive Character Text Splitter	Splits long text into overlapping chunks suitable for embedding.
Default Data Loader	Wraps each chunk together with its metadata into a document object.
Embeddings (Cohere)	Generates a numerical vector embedding for every chunk.
Pinecone Vector Store (Insert)	Upserts the chunk vectors into the firm’s Pinecone namespace, completing indexing.

Figure 4.10 shows the **query workflow**. An AI Agent node orchestrates retrieval and generation: it embeds the question, searches and reranks the relevant chunks in Pinecone, and asks the DeepSeek chat model for a grounded answer. Table 4.2 explains each node.

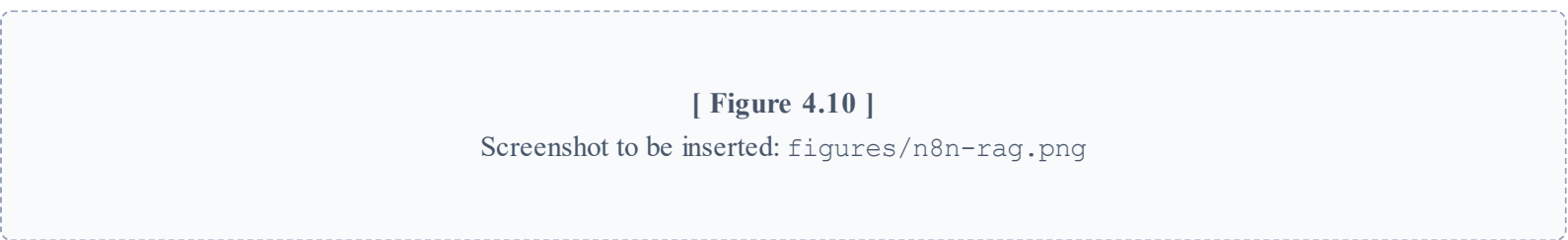


Figure 4.10: n8n Retrieval-Augmented Generation (query) workflow

Table 4.2: Role of each node in the n8n RAG query workflow

Node	Role in the workflow
Webhook (POST)	Receives the user’s question and context from the backend /chat endpoint.
Verify & Extract userId	Validates the request and extracts the user/tenant so retrieval is scoped to the firm.
AI Agent	Orchestrates the RAG loop—decides when to call the retrieval tool and composes the final answer.
DeepSeek Chat Model	The large language model that generates the grounded natural-language answer.
Simple Memory	Maintains short-term conversational context across successive questions.
Pinecone Vector Store (tool)	Retrieval tool; performs semantic search over the firm’s indexed document vectors.
Embeddings (Cohere)	Embeds the question so it can be compared against the stored vectors.
Reranker (Cohere)	Re-orders the retrieved chunks by relevance before they reach the model.
Respond to Webhook	Returns the generated answer (with citations) to the backend, which relays it to the UI.

In the application itself, the assistant is reached through the LexAssist panel (Figure 4.11), which offers guided prompts and renders answers with their supporting citations.

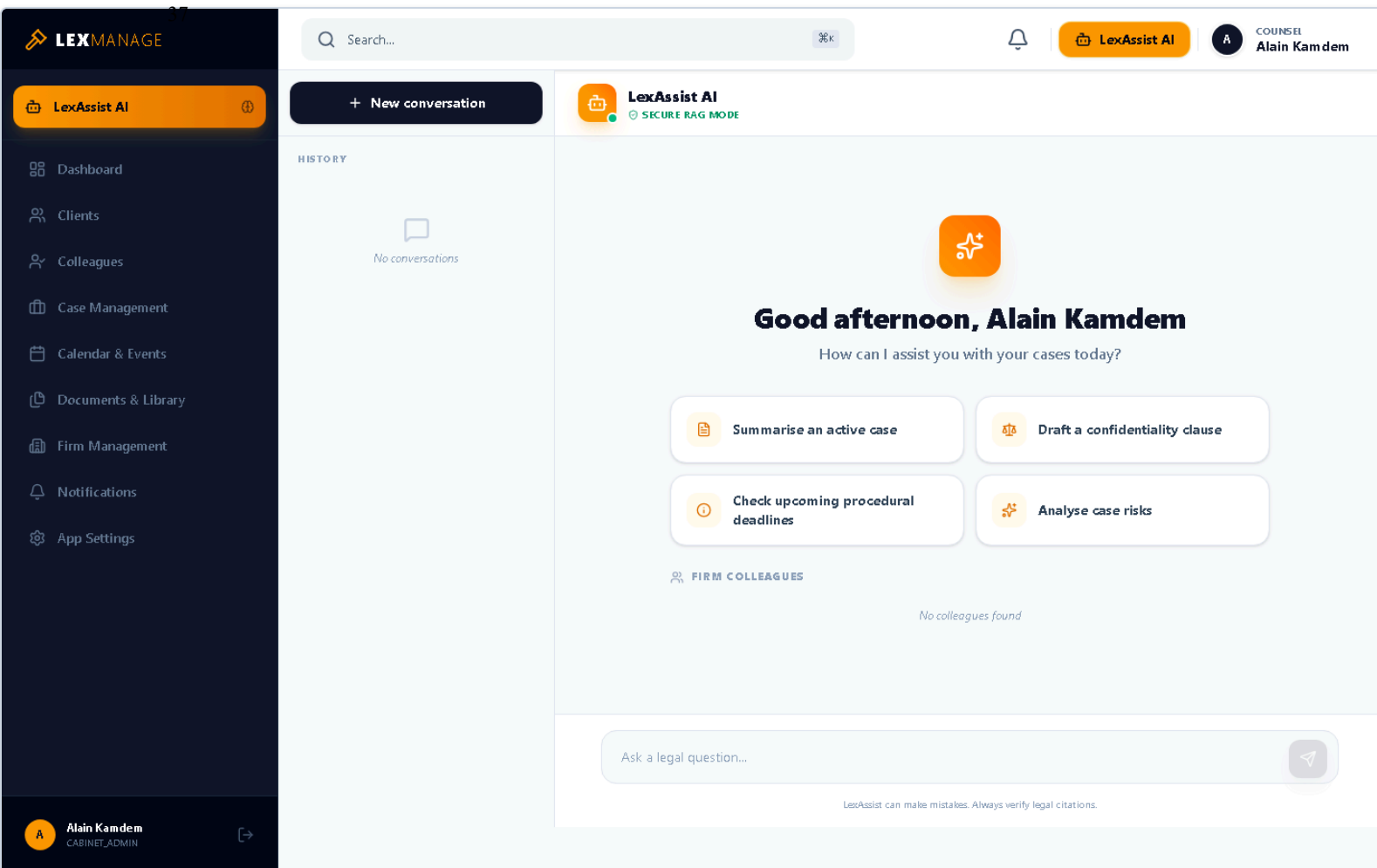


Figure 4.11: LexAssist AI assistant (secure RAG mode) with guided prompts

4.2.7 Account Security and Application Settings

Each user manages their identity and security from the profile view (Figure 4.12), which surfaces multi-factor-authentication status and password management. Application-wide preferences such as language (English/French) and light/dark appearance are configured in settings (Figure 4.13), satisfying the usability and localisation requirements.

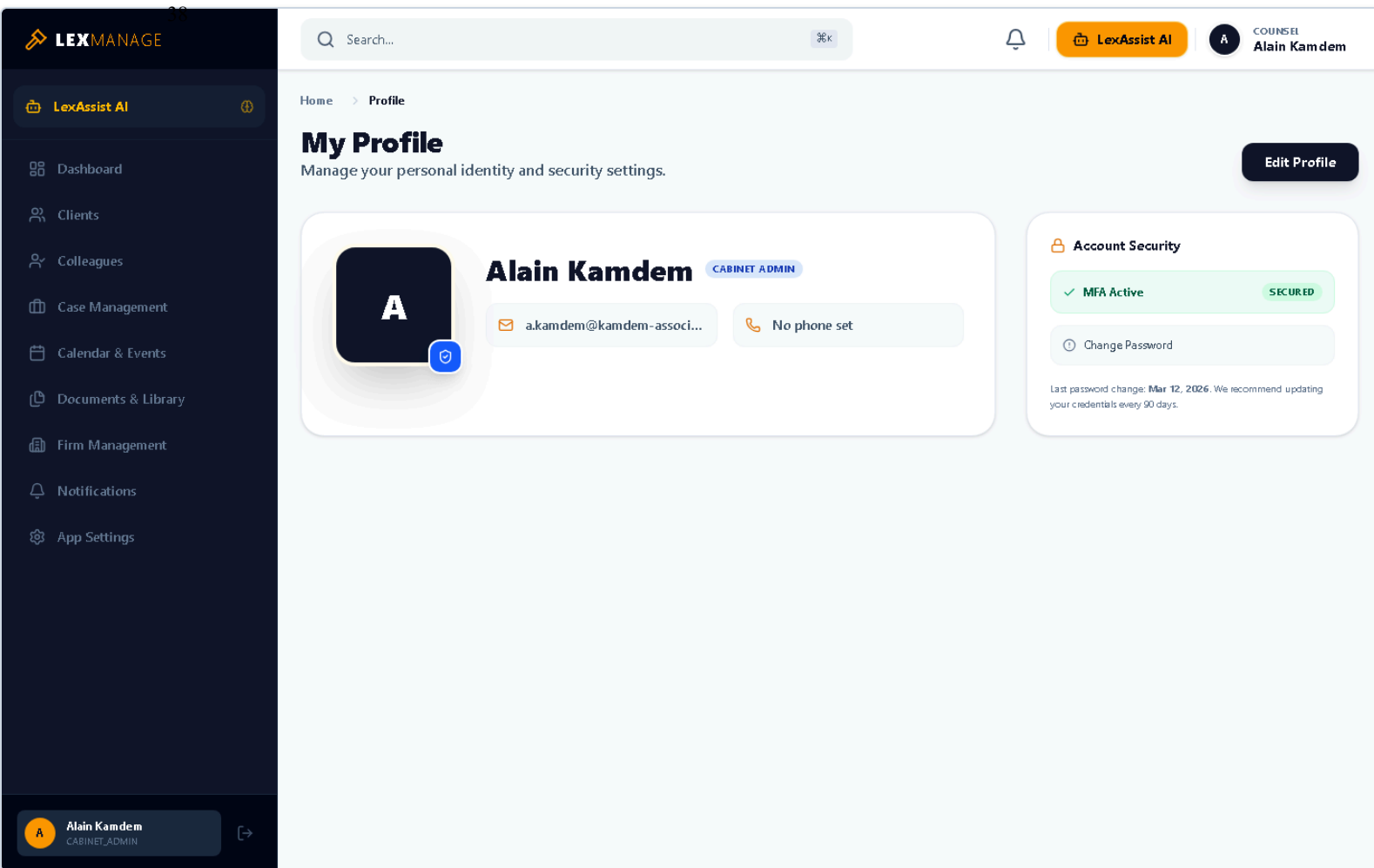
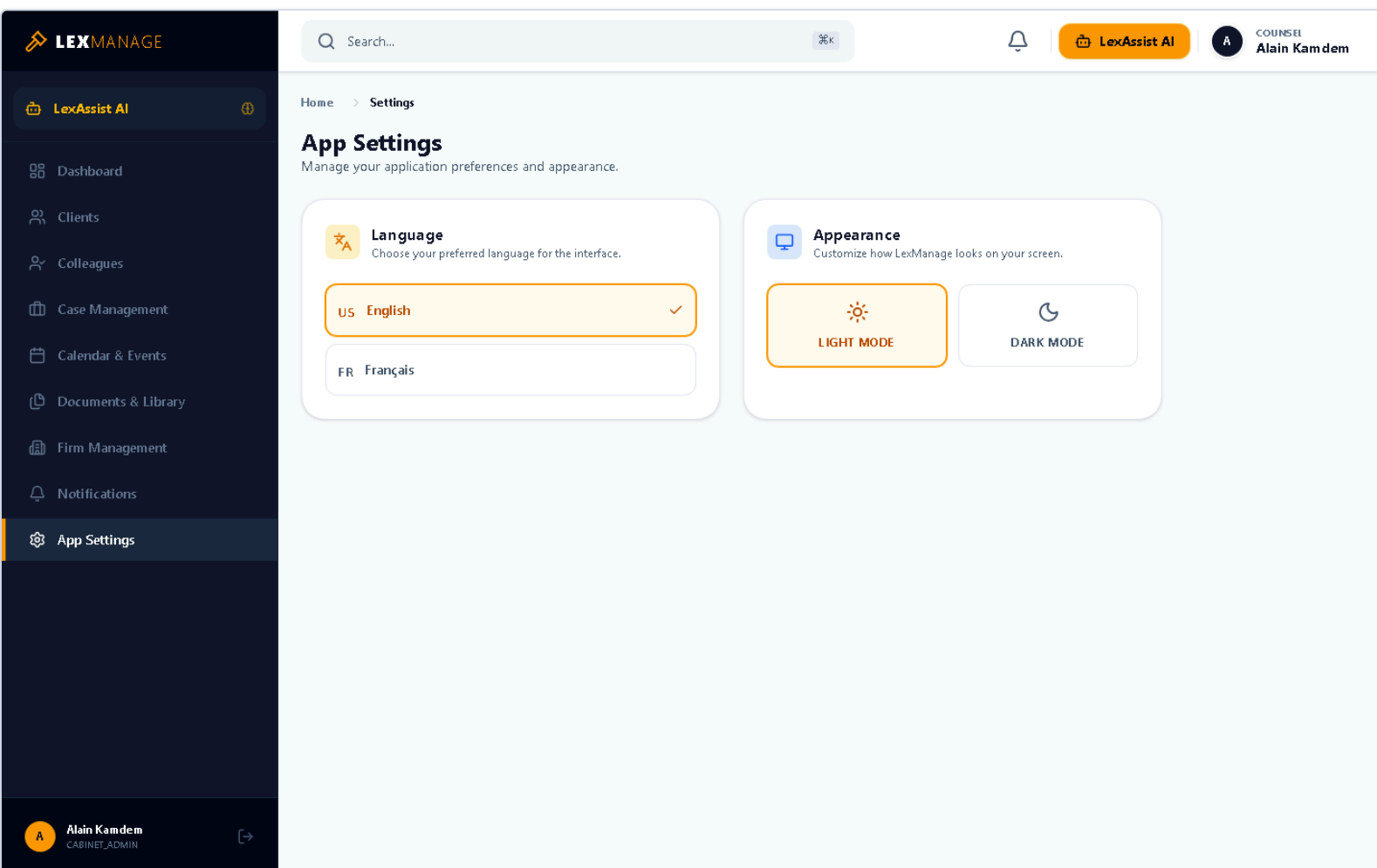


Figure 4.12: User profile and account-security view



LexManage — BTech Software Engineering Project Report
Figure 4.13: Application settings: language and appearance

4.3 Testing

Testing followed the classic V-model levels—unit, integration and system—to validate correctness from individual functions up to complete user workflows. The platform’s resilient design was also confirmed: when a backend service is unavailable, views display clear, non-fatal error states with a retry action rather than crashing.

4.3.1 Unit Testing

Individual services and pure functions were exercised in isolation (for example the notifications service) using the Jest framework, with external dependencies mocked. Unit tests confirmed business rules such as role checks and tenant scoping behave as specified.

Table 4.3: Representative unit test cases

ID	Unit under test	Expected result	Status
UT-1	Password hashing/verification	Correct password verifies; wrong password rejected	Pass
UT-2	Tenant-scoping Prisma extension	Query is filtered by current tenantId	Pass
UT-3	RBAC guard for admin route	Non-admin role is denied access	Pass
UT-4	Notification urgency classification	URGENT alerts flagged for pop-up	Pass

4.3.2 Integration Testing

Integration tests verified that modules cooperate correctly across boundaries—controller to service to database, and API to external services such as MinIO and the n8n AI workflows.

Table 4.4: Representative integration test cases

ID	Interaction	Expected result	Status
IT-1	Login → JWT issue → protected request	Valid token grants access; invalid token rejected	Pass
IT-2	Upload document → MinIO → presigned URL	File stored; time-limited URL returned	Pass
IT-3	Create case → audit log + realtime event	Audit entry written; clients notified live	Pass
IT-4	AI query → n8n (Cohere + Pinecone) → DeepSeek	Grounded answer with citations returned	Pass

4.3.3 System Testing

End-to-end system testing validated complete user journeys against the functional requirements, and confirmed cross-tenant isolation by attempting (and failing, as designed) to access another firm’s records.

Table 4.5: Representative system (end-to-end) test cases

ID	Scenario	Expected result	Status
ST-1	Admin creates firm, invites lawyer, lawyer joins	Both users operate within the same isolated firm	Pass
ST-2	Lawyer creates client, case, schedules hearing	Records persist and reminder is scheduled	Pass
ST-3	User from firm A requests firm B’s case	Access denied; no data leakage	Pass
ST-4	Backend offline during navigation	UI shows graceful error + retry, no crash	Pass

CHAPTER V

CONCLUSIONS AND RECOMMENDATIONS

5.1 Conclusions

This project set out to design and implement a secure, multi-tenant SaaS platform to unify and modernise legal practice management, enhanced with AI document analysis. All of the specific objectives were met: a working platform was delivered with firm onboarding and invitations, JWT authentication and RBAC, strict per-tenant data isolation, full client, case, calendar and document management, real-time notifications, an immutable audit trail, and the LexAssist Retrieval-Augmented Generation assistant. The system was validated through unit, integration and system testing and demonstrates that enterprise-grade isolation and document-grounded AI can be combined in a single, accessible application built entirely on open-source foundations.

5.2 Recommendations

- Adopt a managed PostgreSQL instance and automated backups before any production deployment.
- Add comprehensive end-to-end test automation and a CI/CD pipeline to safeguard future changes.
- Introduce rate limiting and a web-application firewall to harden the public API surface.
- Provide administrator training so firms exploit RBAC, audit logging and the AI assistant fully.

5.3 Perspectives for Further Study

- **Billing & time-tracking:** complete the profitability engine with invoicing and online payments.
- **Native mobile apps:** extend access through dedicated iOS/Android clients.
- **Court e-filing integration:** connect to external judicial systems where APIs exist.
- **Advanced AI:** automatic document classification, clause extraction and deadline detection.
- **Analytics:** richer firm-level dashboards and predictive workload forecasting.

REFERENCES

1. NestJS. *NestJS Documentation — A progressive Node.js framework*. <https://docs.nestjs.com>
2. Meta Open Source. *React Documentation*. <https://react.dev>
3. Prisma. *Prisma ORM Documentation*. <https://www.prisma.io/docs>
4. PostgreSQL Global Development Group. *PostgreSQL Documentation*. <https://www.postgresql.org/docs>
5. Pinecone. *Pinecone Vector Database Documentation*. <https://docs.pinecone.io>
6. n8n. *n8n — Workflow Automation Documentation*. <https://docs.n8n.io>
7. Cohere. *Cohere Embed & Rerank Documentation*. <https://docs.cohere.com>
8. MinIO. *MinIO Object Storage Documentation*. <https://min.io/docs>
9. DeepSeek. *DeepSeek API Documentation*. <https://api-docs.deepseek.com>
10. Lewis, P. et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems (NeurIPS).
11. Socket.IO. *Socket.IO Documentation*. <https://socket.io/docs>
12. Tailwind Labs. *Tailwind CSS Documentation*. <https://tailwindcss.com/docs>
13. Internet Engineering Task Force. *RFC 7519: JSON Web Token (JWT)*. <https://datatracker.ietf.org/doc/html/rfc7519>

APPENDICES

Appendix A — Selected REST API Endpoints

Table A.1: Selected REST API endpoints

Method & Path	Description
POST /auth/login	Authenticate a user and issue JWT access/refresh tokens.
POST /auth/refresh	Exchange a refresh token for a new access token.
GET /cases	List the authenticated firm’s cases (tenant-scoped).
POST /clients	Create a new client in the firm’s directory.
POST /documents	Upload document metadata; returns a presigned URL for the file.
POST /chat	Submit a question to LexAssist; returns a grounded answer with citations.
GET /notifications	Retrieve the user’s notifications.

Appendix B — Source Code Repository

The complete source code of LexManage is maintained under version control at <https://github.com/Emmanuel-Nyouma>. The repository is organised into a React frontend (`src/`) and a NestJS backend (`lexmanage-backend/`) with a Prisma schema describing the data model presented in Chapter III.